POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION

Institute of Computing Science

Bachelor's thesis

AIRCRAFTS DEPLANING OPTIMIZATION

Patrick Molina-Cordero, 157419

Chihabeddine Zitouni, 158763

Marcin Leszczyński, 156061

Mateusz Nowicki, 156064

Supervisor

Dr. Eng. Grzegorz Miebs

POZNAŃ 2026

Contents

List of Acronyms

AAAI	Association for the Advancement of Artificial Intelligence
APU	Auxiliary Power Unit
CA	Cellular Automata
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DES	Discrete-Event Simulation
DI	Dependency Injection
FIFO	First-In, First-Out
GA	Genetic Algorithm
GoF	Gang of Four
HOT	Holdover Time
IoC	Inversion of Control
JAXB	Java Architecture for XML Binding
JDK	Java Development Kit
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MVC	Model-View-Controller
PDP	Passenger Deplaning Problem
PLF	Passenger Load Factor
SA	Simulated Annealing
SFM	Social Force Model
SOP	Standard Operating Procedure
TAT	Aircraft Turnaround Time
TDT	Total Deplaning Time
TS	Tabu Search
WilMA	Window-Middle-Aisle
XML	eXtensible Markup Language

Abstract

Aircraft deplaning is one of the biggest bottlenecks in airport turnaround, mostly because of the chaos caused by passengers grabbing bags and blocking the aisle. For this thesis, a microscopic simulation was built to test exactly how different release strategies affect deplaning speed. It was tested on 720 different scenarios, comparing simple rules like the Zigzag method against complex algorithms like Genetic Algorithms and Tabu Search.

The data showed something unexpected: the most complex algorithms weren't actually the best. Even though the searching algorithms were much slower to run, the simple Zigzag rule was more effective, cutting deplaning time by 71.3% compared to the standard "Front-to-Back" method used by airlines today. This suggests that for a tight space like an airplane, a smart geometric pattern works better than a complex computer search. These results show that airlines can significantly speed up their operations just by changing the deplaning order, without needing any new equipment or expensive software.

Keywords: Aircraft Deplaning, Microscopic Simulation, Metaheuristics, Turnaround Time (TAT), Pedestrian Dynamics, Stochastic Interference, Luggage Factor, Agent-Based Modeling (ABM), Aviation Logistics, Aisle Interference.

Chapter 1

Introduction

The global aviation industry operates on such thin margins that a few minutes' delay can be the difference between a profitable flight and a loss. While the "Smart Airport" concept has successfully optimized baggage handling and security through automation, the aircraft cabin remains as a bottleneck. During deplaning, passenger movement stays largely manual and unoptimized. This thesis explores how we can bridge the gap between high-level computational simulations and the physical constraints of the narrow-body cabin to improve turnaround times (TDT).

In ground operations, efficiency is a major factor for airlines to keep their competitive cost advantage[11]. The Aircraft Turnaround Time (TAT), defined as the duration an aircraft remains parked at the gate between arrival and departure, is a Key Performance Indicator (KPI) [5]. Since an aircraft only generates revenue while in the air, every extra minute an aircraft spends on the ground cuts into the flight's profit margin as fixed operating costs remain constant. With global air traffic rising, there is a huge demand on airports and airlines to speed up these processes. At busy hub airports, high gate occupancy means that a slow deplaning phase doesn't just delay one flight; it stops the next plane from docking, causing taxiway congestion and increased fuel burn. Improving TAT is therefore necessary to maintain schedules on track and get the most out of the fleet [9].

The impact of inefficient deplaning isn't just financial; it also has a clear environmental cost. As the industry faces greater pressure to decarbonize, the ground phase of the fight is being looked at more closely. Longer TAT means the Auxiliary Power Unit (APU) has to run longer to keep the cabin's lighting and climate control systems running. Since APU usage is a significant source of local emissions and noise at airports [1], every minute of delay adds to the flight's carbon footprint before it even leaves the gate. Efficiency is even more critical during winter operations because of de-icing requirements. Before takeoff, aircraft must be de-iced to remove snow and ice, but this process is limited by "Holdover Times" (HOT), the short window during which the de-icing fluid is actually effective [2]. If deplaning is delayed, a plane might miss its de-icing slot or exceed its HOT. This forces the pilot to return for a second, redundant de-icing treatment, effectively doubling the amount of glycol-based chemicals released into the environment and keeping the engines idling longer. Essentially, a slow cabin transition created a chain reaction of waste that extends all the way to the runway.

Processes like refueling, catering, and cleaning are highly standardized and predictable, but passenger deplaning remains one of the most unpredictable parts of the turnaround cycle. Deplaning is often driven by stochastic human behaviors that are hard to control, such as how long it takes different people to retrieve their luggage from the overhead bin or the varying mobility levels of passengers. These individual delays are then made worse by the physical bottleneck of

the narrow-aisle cabin.

Airlines try to control boarding through zone-based ticketing, but deplaning is usually an unregulated process left to social convention and whoever is closest to the exit [14]. These delays don't just postpone the next boarding cycle; they also negatively impact passenger satisfaction. For most travelers, the trip isn't over just because the plane has landed. Instead, sitting in a stopped aircraft while waiting to deplane is often when 'travel fatigue' hits hardest, making it one of the most frustrating parts of the entire passenger experience.

These localized delays often trigger a "ripple effect" (or reactionary delay), where a small 10-minute lag in the morning can snowball into hour-long delays by the airline's final rotation of the day [10]. This bottleneck has a real impact on passenger connectivity. For travelers with tight transfers, the unpredictable nature of deplaning (where the speed of the whole line is often set by the slowest person in the front rows) can easily lead to missed connections.

Even though the turnaround phase is so important, the way we handle deplaning hasn't really changed over the years. Most airlines still just follow a basic 'front-to-back' flow. This rigid approach doesn't account for the stochastic variables that happen on a real flight, like how full the plane is, "Passenger Load Factor" (PLF), people with different mobility needs, or the way the aisle gets blocked when everyone tries to grab their luggage from the overhead bins at the same time.

The core problem is that we lack dynamic, mathematically-based strategies for deplaning. Existing standard operating procedures (SOPs) are too simple to predict or fix the aisle congestions that cause these 'ripple effect' delays. This reveals a clear gap in both the academic research and airlines' operations. There is a need for a simulation-based framework that can model these aisle blockages and determine the optimal flow for different aircraft layouts.

The main goal of this thesis is to develop a simulation framework capable of modeling and optimizing the deplaning process. By treating the aircraft cabin as a system where random human behaviors affect the timing, this research aims to:

- **Identify Key Delays:** Measure how different amounts of carry-on baggage and different cabin layouts change the TDT.
- **Analyze Aisle Bottlenecks:** Model the "interference" that happens when a passenger retrieving their bag stops the flow for everyone behind them.
- **Compare Deplaning Strategies:** Test different exit sequences (such as window-to-aisle or row-based) to see which one is most effective at cutting down TDT.

Work Organization

- Patrick Molina – Thesis drafting, literature review and research, and creation of technical diagrams. Development of advanced optimization algorithms, metrics calculator, experimental execution, and data analysis of results.
- Chihabeddine Zitouni – thesis drafting and literature review and research. Development of basic optimization algorithms, data generation, and experimental execution. Responsible for data analysis of results and simulator visualization.
- Marcin Leszczyński – Lead development and architecture of the simulation environment. Responsible for the execution of code reviews.

- Mateusz Nowicki – Slight contribution to thesis drafting. Development of the random optimizer and auxiliary software functions.

Chapter 2

Theoretical Foundations and Related Work

This chapter reviews the existing research on pedestrian dynamics, aircraft turnaround optimization, and how to mathematically model stochastic flows. While significant research has been dedicated to optimizing passenger boarding, the deplaning phase hasn't received nearly as much attention. By reviewing these different areas, this thesis established a theoretical basis for looking at the cabin as a system where random human behaviors and aisle "interferences" determine the overall speed. We aim to show that these individual delays are the primary drivers of turnaround volatility.

2.1 Pedestrian Dynamics and Microscopic Simulation

The way we model human movement has moved away from looking at crowds as a "fluid" and toward looking at them as individuals. Early research showed that treating people as separate agents with their own goals and ways to avoid collisions is much more effective [6].

When it comes to tight spaces like an aircraft, two main methods are usually used:

- **Social Force Model (SFM):** This looks at people like particles pushed and pulled by "forces," like staying away from walls or moving toward an exit. While it's great for modeling panics, it's often too complex and computationally heavy for just looking at turnaround efficiency.
- **Cellular Automata (CA):** This breaks the space down into a grid where agents move seat-by-seat or row-by-row based on simple rules. This fits the layout of an aircraft cabin perfectly, where movement is already restricted by the rows and seats.

As shown in Figure 2.1, this thesis adopts the Cellular Automata (CA) approach. The cabin is mapped out as a grid where each cell represents either a seat or a section of the aisle. In this model an agent can only move from $cell_t$ to $cell_{t+1}$ if the next cell is empty. This makes the simulation much simpler to manage because we don't have to calculate complex physical collisions; the grid itself handles the movement constraints.

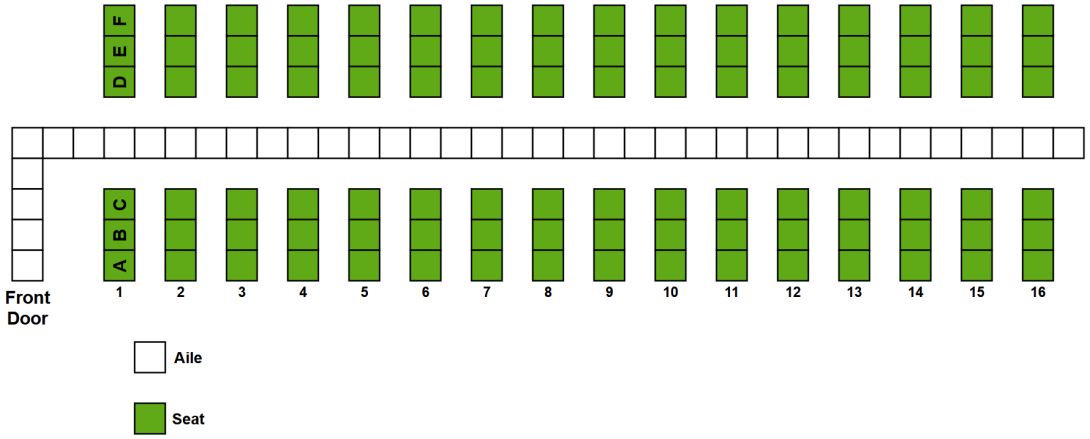


FIGURE 2.1: **Cellular Automata Representation.** A visualization of how the continuous space of an aircraft cabin is discretized into a grid for simulation, where each cell represents a distinct seat (green) or aisle segment (white).

Modern research suggests that for operational analysis, Discrete-Event Simulation (DES) combined with agent-based rules offers the best balance of computational speed and behavioral accuracy [8].

2.2 The Asymmetry of Turnaround: Boarding vs. Deplaning

The academic literature on aircraft turnaround reveals a distinct asymmetry. The vast majority of studies focus on *boarding strategies*, operating under the assumption that airlines have control over the sequence in which passengers enter the aircraft.

2.2.1 Boarding Optimization Strategies

Seminal work by Van Landeghem and Beuselinck utilized simulation to demonstrate that random boarding is often superior to back-to-front strategies due to the "parallelism" of aisle usage [13].

Two major theoretical benchmarks are often cited in this domain:

1. **Window-First (WilMA):** As shown in Figure 2.2, this strategy prioritizes passengers in window seats. The goal is to minimize "seat interference," which happens when a passenger in an aisle seat has to stand up to let someone into the window seat.

2. **The Steffen Method:** Proposed by Jason Steffen (2008), this is often considered the theoretically optimal strategy. It uses a specific row-spacing logic (e.g., boarding seats 30A, 28A, and 26A in sequence) to allow multiple passengers to put away their bags at the same time. By spreading people out along the entire aisle, it aims to stop the bottleneck of everyone crowding the same overhead bins [12].

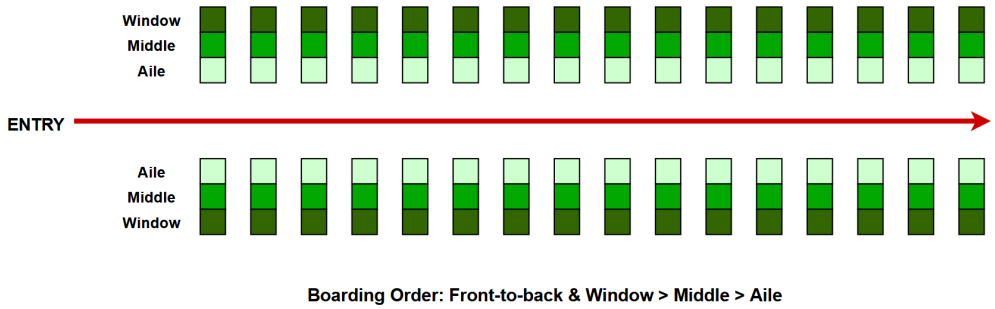


FIGURE 2.2: **Optimization Strategies: Window-First (WilMA).** A standard optimization baseline where passengers are grouped by column (Window, Middle, Aisle). The theoretical Steffen Method extends this logic by interleaving rows to maximize bin access.

2.2.2 The Deplaning Gap

In many models, deplaning is treated as just "boarding in reverse," which ignores a key difference: how the crowd behaves. During boarding, passengers arrive at the aisle one by one, but during deplaning, almost everyone stands up at the same time. This creates immediate aisle congestion that doesn't happen during the boarding phase.

Recent studies have started to look into this. For example, research using CA models has shown that using both the front and rear doors for egress can cut deplaning time by up to 30% [8]. However, a lot of this research still relies on "ideal" passenger behavior. They don't always fully account for the random delays caused by people struggling to get their carry-on bags out of the overhead bins.

2.3 Stochastic Interference and the "Luggage Factor"

The main bottleneck during deplaning isn't actually how fast people walk, but rather "interference." This happens whenever a passenger stops the flow in the aisle to get their bags out of an overhead bin. In theoretical models, this is defined as an *Interference Radius*: the physical space that is blocked off while a passenger is busy retrieving their luggage.

We define two distinct types of interference that must be modeled:

- **Type 1 Interference (Seat-Aisle Conflict):** This occurs within the row itself. As shown in Figure 2.3, a passenger in a "blocked" seat (e.g., Window/Black) cannot enter the aisle until the "blocking" passengers (e.g., Aisle/Red) have cleared the space. In high load-factor scenarios, this creates a serialization of movement within the row group.

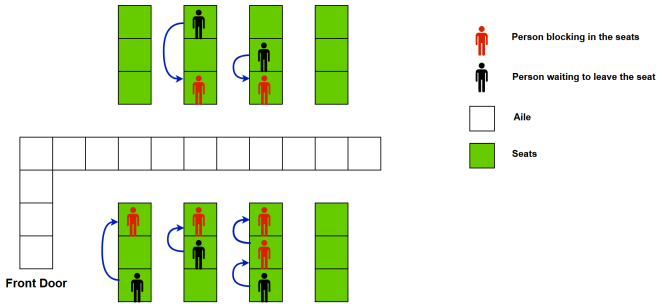


FIGURE 2.3: **Type 1 Interference (Seat Conflict).** A high-priority passenger (Black) is physically obstructed from accessing the aisle by lower-priority passengers (Red). This necessitates a sequential "peeling" maneuver before egress can begin.

- **Type 2 Interference (Aisle-Blocking):** This is the dominant variable in Total Deplaning Time (TDT). As visualized in Figure 2.4, when a passenger enters the aisle and pauses to retrieve luggage ($t_{retrieval} > 0$), they occupy the aisle resource. This creates a hard blockage, reducing the velocity of all upstream passengers ($Row > n$) to zero.

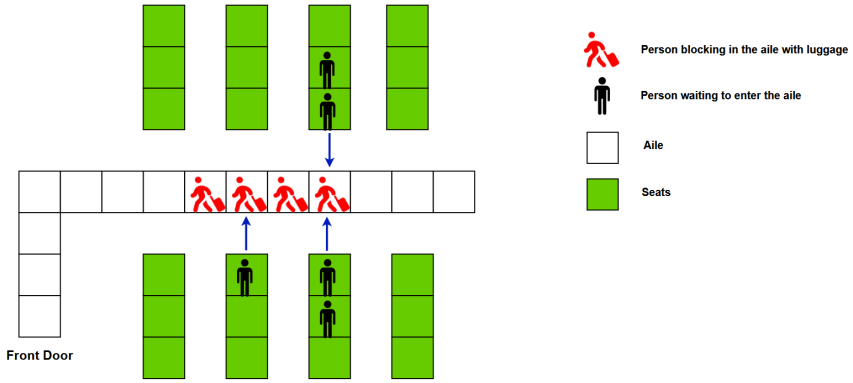


FIGURE 2.4: **Type 2 Interference (Aisle Congestion).** The "Ripple Effect" mechanism. A single passenger retrieving luggage (Red) halts the downstream flow. The queue of passengers behind (Black) cannot advance until the retrieval action is complete.

Recent analyses of time series in air transport highlight that these delays are not linear but can trigger "ripple effects," where small local delays cascade into significant operational disruptions [10]. Current literature lacks a dynamic optimization model that specifically targets minimizing these interferences through sequenced release strategies.

2.4 Mathematical Optimization in Logistics

To solve the combinatorial problem of deplaning sequences, researchers use metaheuristic algorithms. Since exact methods like Linear Programming can't handle the scale of $N!$ permutations, metaheuristics are used to find "good enough" solutions within a realistic time frame.

- **Genetic Algorithms (GAs):** These are useful for evolving strategies in which different "traits," such as releasing window seats before aisle seats, can be combined over several generations to find the best overall sequence.
- **Simulated Annealing (SA):** This method is inspired by thermodynamics and is great for "escaping local optima" [7]. For deplaning, SA allows the simulation to occasionally test a "slower" sequence to see if it eventually leads to a much faster pattern that clears the aisle better.
- **Tabu Search (TS):** This uses "memory lists" to stop the algorithm from wasting time on solutions it has already tried [3]. This is especially helpful in our case; if swapping Row 10 and Row 11 doesn't change the deplaning time, the "tabu list" forces the algorithm to explore completely different row combinations instead of getting stuck in a loop

2.5 Summary and Research Gap

While previous research has established the basic physics of pedestrian flow and the standard metrics for boarding, a clear gap remains in the literature. Most existing models either:

1. Focus almost entirely on boarding and ignore the specific constraints of the deplaning phase.
2. Use static values that don't account for how much carry-on baggage actually changes the flow in real-time.

This thesis addresses these issues by developing a simulation framework that focuses on baggage-related delays. By using metaheuristic algorithms, we aim to find more efficient deplaning sequences that might not be obvious through standard observation.

Chapter 3

Methodology: Computational Framework

To address the optimization challenges mentioned in the Introduction (specifically the stochastic nature of passenger movement and the "ripple effect" of deplaning delays), this thesis utilizes a custom-built DES engine. This chapter explains the technical architecture, software stack used for development, and specific algorithms employed to model the aircraft cabin as a dynamic system.

3.1 Software Architecture and Technology Stack

The simulator is built using a modular software stack to ensure reproducibility, scalability, and precise control over passenger behavior. The framework allows for different deplaning strategies (e.g., Back-to-Front vs. Random) to be tested quickly without needing to rewrite the core engine. This flexibility is essential for comparing various operational scenarios under consistent conditions.

3.1.1 Core Language and Build System

- **Java (Java Development Kit (JDK) 17+):** Java was selected as the primary implementation language. Beyond its object-oriented capabilities, Java's memory management and thread safety are critical for maintaining the state of hundreds of independent Passengers.

  Relevance to Thesis: Modeling the stochastic nature of deplaning requires a "State Machine" approach for each passenger. Java's strict type system ensures that state transitions such as moving from an "Aisle Flow" state to a "Baggage Interference" state are handled without logic errors. This ensures that the simulation results accurately reflect the physical constraints of the cabin rather than software artifacts.

- **Gradle:** Gradle is employed as the build automation and dependency management tool. It handles the multi-module architecture of the project, isolating the core simulation logic from the data-processing and visualization layers.

  Relevance to Thesis: In scientific modeling, reproducibility is paramount. Gradle allows for the definition of a deterministic environment where all libraries and configurations are locked. This ensures that the TDT (Total Deplaning Time) results can be reproduced across different hardware environments, which is essential for verifying the statistical significance of the deplaning strategies tested.

3.1.2 Application Framework

- **Spring Framework (Core & Inversion of Control (IoC)):** Although Spring is a standard tool for building web applications, the simulator incorporates its IoC (Inversion of

Control) and Dependency Injection (DI) (Dependency Injection) features to manage the simulation's settings.

Relevance to Thesis: Using Spring makes the simulator much more flexible for testing different scenarios. Instead of hard-coding the rules, variables like "Cabin Layout," "Passenger Distribution," and "Baggage Probability" are injected into the engine at runtime. This allows for the evaluation of completely different setups like comparing a full flight to a half-empty one just by swapping a configuration file. This keeps the core movement code clean and makes it easier to run the high number of tests needed for the results chapter.

3.1.3 Code Optimization and Utilities

- **Project Lombok:** This is a library that automatically generates "boilerplate" code, such as getters, setters, and constructors, during the build process.

  Relevance to Thesis: Because Java requires a lot of repetitive code for simple data objects, Lombok was used to keep the source files clean and easy to read. This allowed development to remain focused on the actual simulation logic—like the rules for passenger movement—rather than on maintaining hundreds of lines of basic Java syntax.

- **Google Guava:** This library is used for its immutable collection types and specialized utilities.

  Relevance to Thesis: Maintaining data integrity is vital for a scientific simulation. The model employs Guava's immutable collections to ensure that once a cabin layout or passenger list is initialized, it cannot be accidentally changed during the simulation run. This prevents "data drift" and ensures that the final results are based on the exact parameters intended at the start.

- **Apache Commons Lang:** This is a collection of utility tools that help with common programming tasks like string manipulation and object comparison. ***Relevance to Thesis:*** During the testing phase, it was essential to accurately track and debug the state of different passengers. These utilities were used to create clear, detailed logs of simulation events, which helped in verifying that the "interference" and "blocking" logic was working correctly before running the final experiments.

3.1.4 Data Serialization and Analysis

- **Jackson (JavaScript Object Notation (JSON) Processor) & Java Architecture for XML Binding (JAXB) (eXtensible Markup Language (XML) APIs):** hese libraries are used to convert simulation data into formats that are easy to read and process. Jackson is used to load the simulation settings from JSON files, while JAXB exports the data needed for visualization into XML format.

  Relevance to Thesis: These tools facilitate the conversion of raw calculations into into a visual format. By exporting a snapshot of the cabin at every time step (t), it becomes possible to see and analyze the exact bottlenecks that were identified in the problem statement.

- **JUnit 5 & Spring Test:** This is the testing framework used to verify that the simulator's logic is working correctly.

  Relevance to Thesis: To ensure the results are scientifically valid, the core rules of the model must be verified. Unit tests are utilized to verify that the foundational rules such

as the requirement that two passengers cannot stand in the same spot at once—are strictly followed before any complex deplaning scenarios are simulated.

3.2 Algorithmic Design and Patterns

To model the complex "interference patterns" mentioned in the introduction, the software employs several Gang of Four (GoF) design patterns. These patterns transform the theoretical "Bottleneck" problem into manageable software components.

3.2.1 The Decorator Pattern: Modeling Complexity

To manage the "stochastic human behaviors," actions are decoupled from the agents.

- **Concept:** In object-oriented design, a Decorator allows behavior to be added to an individual object, dynamically, without affecting the behavior of other objects from the same class.
- **Implementation:** The system defines a standard agent that handles basic walking movement. A specialized "baggage-handling" layer then wraps this base behavior for specific individuals.
- **Application:** When the simulation initializes, it stochastically assigns these layers. A passenger might behave as a standard traveler, or they might be assigned the additional baggage retrieval traits. This architecture allows for future expansion—such as layering on reduced movement speeds for elderly passengers—directly addressing the research objective of "Quantifying Variable Impact."

3.2.2 The Strategy/Command Pattern: Discrete Actions

To manage random human behaviors, specific movements are separated from the passenger agents themselves. This allows this thesis to treat every physical effort as an independent event.

- Concept: Every task a passenger performs is treated as a distinct object. This makes it possible to assign different instructions to various passengers without changing the core rules of the simulation.
- Implementation: Every physical step—such as moving forward, stepping into the aisle, or reaching for a bag—is modeled as an individual "Action" unit.
- Application: This design allows the simulation to mimic the hesitation and timing differences found in real life. An action is not finished instantly; it has its own duration and a "progress state." This level of detail allows the simulator to measure exactly how long a specific blockage lasts, providing the data needed to analyze the "Ripple Effect" of deplaning delays.

3.2.3 The Locking Mechanism: Solving Spatial Conflict

A critical component of the simulator is how it manages the aisle as a shared resource.

- The Problem: In the narrow aisle of an aircraft, space is limited. As noted in the Problem Statement, interference occurs when passengers from different rows all try to access the same section of the aisle at once.

- The Solution: The simulation uses a logic similar to a "reservation system" for movement. Before a passenger
    1. **Check:** The agent checks if the next position in the aisle is currently empty.
    2. **Reserve:** If it is empty, the agent "locks" that spot so no other passenger can claim it.
    3. **Move:** The agent performs the movement into the new space.
    4. **Release:** The agent clears their previous position, making it available for the passenger behind them.
- **Relevance:** This logic prevents "clipping" (where two passengers occupy the same space) and accurately mimics the stop-and-go waves that happen during real-world deplaning.

3.3 Simulation Logic Flow

The engine operates on a discrete-time loop, where each "tick" represents exactly one second of real-world time. The execution flow is designed to capture how the cabin environment changes second-by-second:

1. **State Evaluation:** Every passenger evaluates their current surroundings. This includes checking their distance to the exit, the presence of their own luggage, and the position of the passenger directly in front of them.
2. **Action Selection:** Based on their assigned behavioral layers—such as whether they are carrying heavy bags—each agent selects the most appropriate next step. This might be moving forward, standing up, or initiating a baggage retrieval delay.
3. **Conflict Management:** The simulation engine acts as an arbitrator to manage the limited space. If two passengers attempt to move into the same spot at the same time, the engine resolves the conflict using priority rules. This mimics the "social conventions" of deplaning, such as giving precedence to those already moving in the aisle.
4. **Recording and Data Capture:** At the end of every second, the exact position and status of every person in the cabin are recorded. This creates a detailed dataset that allows for the visual reconstruction of the deplaning process and provides the raw numbers needed for the final performance analysis.

Chapter 4

Optimization Algorithms for Passenger Deplaning

4.1 Introduction

The Passenger Deplaning Problem (PDP) is essentially the inverse of the boarding problem, yet it presents several unique constraints. While airlines can control boarding through gate sequencing, deplaning is usually a "free-for-all" governed by social conventions and the physical bottleneck of the forward exit. The primary objective is to minimize the TDT by finding an optimal "Release Sequence." This refers to the specific order in which groups of passengers are allowed to stand up, retrieve their luggage, and enter the aisle.

Mathematically, the problem is treated as a search for permutation sequence $S = \{G_1, G_2, \ldots, G_n\}$ of passenger groups that minimizes the fitness function $f(S)$, which is defined as the total number of simulation "ticks" required for the cabin to empty.

4.2 Methodological Framework

The optimization framework employs a modular design that enables the seamless integration of various deplaning algorithms with the core DES engine. This structure ensures that the logic governing passenger flow remains distinct from the simulation mechanics, allowing for the flexible testing of different deplaning strategies.

4.2.1 The Optimization Logic Layer

The central layer of the framework serves as the interface between the mathematical models and the simulation environment. This layer coordinates the optimization process through three primary phases:

1. **Sequence Generation:** Formulating the specific order of passenger groups based on predefined criteria, such as row-based or seat-type priority.
2. **Simulation Execution:** Implementing this generated sequence within the simulation environment to determine the total deplaning duration, denoted as T_{deplane}.
3. **Data Recording and Evaluation:** Capturing the most efficient sequences and their corresponding performance metrics to facilitate comparative analysis and post-simulation review.

This structure is divided into two main types of implementations:

- **Deterministic Heuristics:** These are algorithms that follow fixed, "common sense" rules, such as "Aisle seats first" or "Front-to-Back." They always produce the same result for a given cabin layout.
- **Stochastic Metaheuristics:** These are more advanced algorithms that use probabilistic search techniques. They explore many different combinations to find efficient release sequences that might not be obvious through simple observation.

4.3 Deterministic Heuristics

These algorithms represent the standard or theoretical baseline strategies. They rely on fixed logical rules rather than iterative searching, acting as the "control group" for the experiments in this thesis.

4.3.1 Row-Based Strategies (Front-to-Back)

The Standard Operating Procedure (SOP) in commercial aviation is an uncontrolled **Front-to-Back** flow. In most flights, this is the default behavior as passengers naturally follow the person in front of them.

Operational Logic: The cabin is divided into row segments $S = \{G_1, G_2, \ldots, G_n\}$.As seen in Figure 4.1, passengers in row r_i generally wait for row r_{i-1} to clear before they can move. The cabin is treated as a single queue where the "active" zone moves slowly from the front to the back of the aircraft.

FIGURE 4.1: **Front-to-Back Strategy Visualization.** The cabin is divided into contiguous row blocks. The strategy enforces a strict sequential dependency: Block N (green) must complete egress before Block $N+1$ (yellow) initiates movement, maximizing aisle idleness in the rear sections.

Theoretical Limitation: This method maximizes "aisle interference". As passengers in row r_i stand to retrieve overhead luggage, they physically block the aisle for anyone behind them. The process becomes serial rather than parallel [13]. The inefficiency is clearly depicted in the simulation: the "ripple effect" of a single stuck bag creates a complete stoppage for all upstream rows.

4.3.2 Aisle-First Strategy (Reverse WilMA)

This strategy tries to minimize "seat interference" (the delay caused when an aisle passenger must wait for a window passenger, or vice versa). For deplaning, the optimal logic is the inverse of the boarding "Window-Middle-Aisle (WilMA)" method.

Operational Logic: The set of passengers P is divided into three subsets based on column index: $S_{\text{aisle}} \rightarrow S_{\text{middle}} \rightarrow S_{\text{window}}$.

As shown in Figure 4.2, this strategy "peels" the cabin from the inside out. All aisle passengers deplane first, effectively widening the available maneuvering space for the subsequent middle and window passengers. This topological ordering ensures that no passenger is ever physically blocked by a seatmate.

FIGURE 4.2: **Aisle-First (Reverse WilMA) Priority Map.** Passengers are grouped by column rather than row. Phase 1 (Green/Aisle) clears the central pathway. Phase 2 (Yellow/Middle) and Phase 3 (Red/Window) follow sequentially. This reduces row-internal friction but risks high density in the main aisle.

Performance Trade-off: While theoretically faster due to reduced friction between neighbors, this method can suffer from congestion if too many aisle passengers attempt to exit simultaneously. The simulation often shows that it can cause a "density spike" in the aisle during the first phase, shifting the bottleneck from the seats to the walkway.

4.3.3 The ZigZag Strategy (Reverse Steffen)

This heuristic is an adaptation of the optimal boarding method proposed by Jason Steffen [12], applied in reverse for exit. It aims to maximize parallel overhead bin access.

Operational Logic: Passengers are released in a staggered pattern (e.g., 1A, 3A, 5A...).The main goal is to separate the "Interference Radius" of each person. Figure 6.3c demonstrates this alternating pattern: by ensuring that no two adjacent rows are active simultaneously, the algorithm guarantees that every active passenger has sufficient physical space to lower their luggage without blocking a neighbor.

FIGURE 4.3: **ZigZag (Reverse Steffen) Release Pattern.** Passengers are released in an interleaved sequence (Row N, Row $N+2$, etc.). This checkerboard activation pattern ensures that the 'Luggage Retrieval Radius' of one passenger never overlaps with another, theoretically allowing for maximum parallel flow.

4.4 Advanced Metaheuristic Optimizers

While the basic heuristics provide a good starting point, they are "static" and cannot adapt to random factors like heavy baggage loads or different walking speeds. To find the best possible deplaning sequence (the global optimum), this thesis uses three different metaheuristic algorithms.

4.4.1 Genetic Algorithm (GA)

The GA treats the deplaning sequence like a biological evolution problem [4]. It starts with a "population" of different release sequences and "evolves" them over many generations to see which ones result in the fastest times.

Evolutionary Logic: As shown in Figure 4.4, the algorithm keeps a "gene pool" of strategies. Through "Tournament Selection" and "Crossover," the best traits such as releasing specific row combinations together are passed on to the next generation. We also use "Mutation" to randomly change some sequences, which stops the algorithm from getting stuck on one acceptable solution and encourages it to find even better ones.

- **Encoding:** A solution is represented as a permutation of passenger groups.
- **Operators:**
  - **Tournament Selection:** Selection of k individuals. In this implementation a subset of size $k = 3$ is selected, and the fittest individual becomes a parent.
  - **Order Crossover (OX1):** Preserves the relative ordering of a sub sequence from Parent A while filling remaining slots from Parent B.
  - **Mutation:** With probability P_m, in this implementation 33% each, the algorithm applies structural changes:
    - **Merging:** Combining two random groups → reducing the total groups.
    - **Split:** Divide random group at random point → increasing granularity.
    - **Reverse:** Flip order of sub sequence of groups → exploring different release orders.
  - **Elitism:** Always preserve the best individual to prevent fitness regression.
  - **Validity Checking:** Each offspring is validated to ensure all passengers appear exactly once.
- **Fitness Function:** The fitness is defined directly as the simulation time T.

$$\text{Fitness}(x) = \text{SimulateDeplaning}(x)$$

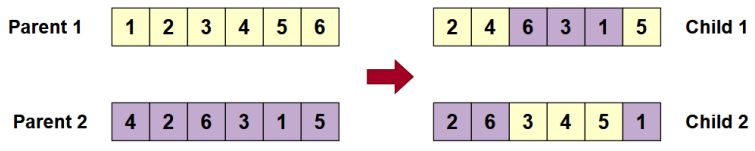


FIGURE 4.4: **GA Evolutionary Cycle.** Two parent sequences undergo 'Order Crossover' to produce offspring. This mechanism preserves the relative ordering of high-performing sub-sequences (traits) from Parent A while filling the remaining slots with genetic material from Parent B, ensuring valid permutations.

4.4.2 Simulated Annealing (SA)

SA is a probabilistic technique capable of escaping local optima (e.g., a sequence that is locally fast but globally inefficient).

- **Process:** The system starts with an initial temperature T_0 and a random release sequence S.
- **Acceptance Probability:** If a new sequence S' results in faster deplaning ($\Delta E < 0$), it is accepted. If it is slower, it is accepted with probability:

$$P(\text{accept}) = e^{-\Delta E/T}$$

  where T decays geometrically via a cooling rate α. This allows the algorithm to explore "worse" strategies early on to find novel patterns (e.g., releasing the back of the plane first to clear a bottleneck).

4.4.3 Tabu Search (TS)

TS utilizes memory structures to guide the local search for an optimal release sequence. Unlike annealing which uses randomness to escape local optima, TS uses deterministic memory.

Memory-Guided Search: Figure 4.5 outlines the decision logic. A "Tabu List" acts as a short-term memory, recording recent moves (e.g., "Swapped Row 10 & 11"). If the algorithm attempts to reverse this move within N iterations, the action is blocked unless it satisfies the "Aspiration Criterion" (i.e., it finds a new global best). This forces the search trajectory to explore new regions of the solution space rather than cycling endlessly around a local minimum.

- **Tabu List:** The algorithm maintains a memory of recently tested sequences to prevent cycling. It uses Fixed size First-In, First-Out (FIFO) queue to store these solution representations.
- **Aspiration Criterion:** A "tabu" (forbidden) move is permitted if it results in a deplaning time better than the global best found so far. Preventing getting stuck when the tabu list blocks the path to better solutions.
- **Relevance:** This is particularly effective for deplaning, where small changes in sequence (e.g., swapping Row 10 and Row 11) often result in identical times. The Tabu list forces the algorithm to try radically different sequences.

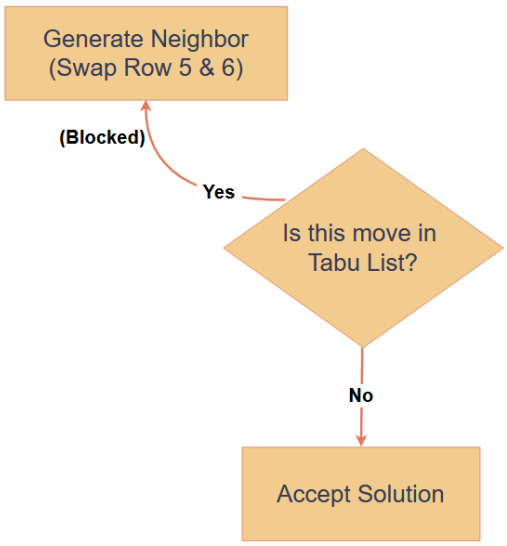


FIGURE 4.5: **TS Decision Flowchart.** The algorithm evaluates neighbor solutions against a 'Tabu List' of forbidden moves. The 'Aspiration Criterion' acts as a bypass switch: if a forbidden move yields a global optimum, the Tabu restriction is ignored, ensuring that the best possible solution is never discarded due to memory constraints.

4.5 Optimization Process and Methodology

The optimization workflow integrates tightly with the DES engine.

4.5.1 Fitness Evaluation Constraints

To make the optimization computationally feasible, the evaluation process is bifurcated:

- **Fast-Path Evaluation:** During the iterative search, the simulation runs in "headless" mode. Visualization rendering and XML serialization are disabled. This reduces the evaluation time per solution from $\approx 500\,\text{ms}$ to $\approx 15\,\text{ms}$.
- **Full-Path Validation:** Only the final optimal sequence is subjected to a full simulation run with visualization enabled, generating the data needed for analysis.

4.5.2 Neighborhood Operators

The system defines four atomic operations to transition between potential release strategies:

1. **Merge:** Combines two passenger groups into a single release signal.
2. **Split:** Divides a group into two phases.
3. **Swap:** Exchanges the priority of two groups.
4. **Move:** Relocates a group to a new position in the priority queue.

4.6 Algorithm Comparison and Complexity

Table 4.1 summarizes the computational complexity of the implemented strategies, where N is the number of passengers. For the metaheuristics, P denotes the population size, G is the number of generations, I represents the total iterations, and K is the neighborhood search size.

Algorithm	Time Complexity	Search Space	Characteristics
Front-to-Back	$O(N)$	Single Point	High Aisle Interference
Aisle-First	$O(N)$	Single Point	Low Seat Interference
GA	$O(P \times G \times N)$	Population-based	Good for large, complex cabins
SA	$O(I \times N)$	Trajectory	Escapes local optima
TS	$O(I \times K \times N)$	Trajectory	Prevents cycling in flat landscapes

Table 4.1: Computational Complexity of Deplaning Strategies

4.7 Summary

This chapter has established the algorithmic tool kit used to solve the deplaning optimization problem. By implementing both industry-standard heuristics and advanced metaheuristics, the framework is capable of simulating current realities (Front-to-Back) and discovering novel release strategies that minimize turnaround time.

Chapter 5

Simulator

This chapter provides a detailed look at how the simulation engine actually works. Now that the software structure and design patterns have been established, the focus moves from the technology used to the way the simulation is executed. This thesis explains the specific mechanisms the simulator uses to turn theoretical deplaning strategies into measurable data, showing how passenger movement and delays are calculated step-by-step.

5.1 Core Simulation Architecture

The simulation engine works as a step-by-step system where time moves forward in single units, or "ticks." In every tick, each passenger in the cabin checks their surroundings and tries to perform their next move based on their current situation and the physical limits of the aircraft.

5.1.1 The Simulator Service: Central Orchestration

The main controller of the simulation manages three essential parts of the model:

- **Plane Geometry:** The physical map of the cabin, which defines where the seats, aisles, and exits are located.
- **Passenger Agents:** The individual people in the simulation, each with their own location, behavior traits, and planned actions.
- **Temporal Controller:** The mechanism that moves the simulation forward second-by-second, making sure that actions happen in the correct order and that two people don't try to do something impossible at the same time.

Relevance to Thesis: This organized structure allows the "ripple effect" of passenger delays to be tracked with high precision. If a passenger blocks the aisle at a specific second, the resulting delay for everyone behind them is captured exactly as it happens. This provides the data needed to see how a single slow action creates a much larger problem for the entire deplaning process.

5.1.2 Simulation Initialization: Problem Construction

Before the simulation starts, a "deplaning scenario" is created. This setup defines all the starting conditions for the test run:

1. **Passenger Groups:** Passengers are organized into groups based on the specific deplaning strategy being tested. For example, they are grouped by row numbers for a "Front-to-Back" test, or by seat position (Aisle, Middle, or Window) for a "Reverse WilMA" test.

2. **Initial Placement:** Every passenger in the model is created with specific characteristics:

   - **Seat Location:** Their starting coordinates $(row, column, side)$.
   - **Speed Profiles:** Their walking speed in the aisle and their movement speed when sliding out of a seat.
   - **Baggage Status:** A flag indicating whether the passenger has overhead luggage and how many seconds it will take them to retrieve it.

Key Configuration Parameters:

- **Aisle Speed:** The base speed at which a passenger walks down the aisle.
- **Unobstructed Speed:** The speed used when a passenger is moving toward the aisle without being blocked by neighbors.
- **Baggage Delay:** The extra time added to a passenger's progress when they have to stop and retrieve a bag from the overhead bin.

5.2 Discrete-Time Execution Loop

The core simulation loop follows a structured, repeating cycle. During every time unit, every passenger currently on the aircraft is given the opportunity to act. This process continues until the cabin is entirely empty. The pseudocode structure is:

```
1: while activePassengers > 0 do
2:     currentTick ← currentTick + 1
3:     for all passenger ∈ plane.getPassengersOnSeats() do
4:         selectedAction ← passenger.chooseAction()
5:         actionResult ← selectedAction.execute()
6:         updatePassengerState(passenger, actionResult)
7:     end for
8:     resolveConflicts()
9:     captureFrameData()
10: end while
```

5.2.1 Action Selection Mechanism

Each passenger evaluates their surroundings to determine their next move. This decision process follows a specific priority list, ensuring that the agents behave according to physical constraints and the chosen deplaning strategy.

1. **Completion Check:** The agent first checks if they have reached the aircraft exit. If so, they are removed from the active simulation and marked as "deplaned."

2. **Baggage Retrieval:** If a passenger has overhead luggage that hasn't been picked up yet, they must initiate a retrieval action. This creates a mandatory delay (the "Baggage Delay" parameter) that prevents the passenger—and those behind them—from moving forward.

3. **Aisle Entry:** If a passenger is still at their seat but is now allowed to leave (according to the "Release Sequence"), they check if the aisle space is clear. If available, they move from the seat row into the aisle.

4. **Forward Movement:** For passengers already in the aisle, the priority is to move toward the exit. The agent checks if the spot directly in front of them is empty before taking a step forward.

5. **Waiting:** If none of the above actions are possible—usually because the aisle is crowded or the person in front is still retrieving luggage—the passenger enters a "Wait" state for that second.

Mathematical Representation:

The state of a passenger i at any given second t by $S_i(t)$. The selection of an action A is a function of the passenger's current state and the environment E E(t) (which includes aisle occupancy and distance to the exit):

$$A : S_i(t) \times E(t) \rightarrow \{MoveForward, EnterAisle, RetrieveBaggage, Wait\}$$

where $E(t)$ encodes the environmental state (aisle occupancy, exit proximity).

5.2.2 Action Execution and State Transitions

Once an action is selected, the engine executes it and determines the outcome. Every action results in a specific status update for the passenger, which includes:

- **Completion Status:** A check to see if the action was successful. For example, if a passenger tries to step forward but the space is blocked, the action is marked as failed, and the passenger remains in their current spot.

- **Updated Coordinates:** If a movement action is successful, the passenger's location is updated to their new position in the cabin grid.

- **Time Duration:** The number of seconds (ticks) the action takes to complete. While a standard step forward takes 1 second, retrieving a bag will take a much longer duration based on the "Baggage Delay" parameter.

This setup allows the simulator to model a mixed population where only some passengers are delayed by baggage. By applying these rules selectively, this research can precisely quantify how much impact individual luggage delays have on the total deplaning time.

5.3 Spatial Conflict Resolution: The Queue System

The simulation treats the aircraft aisle as a sequence of individual, fixed positions. This approach directly addresses the physical constraint that no two passengers can occupy the same spot at the same time.

5.3.1 Movement and Reservation Protocol

To ensure realistic movement, the simulation uses a "reservation" system. Before a passenger moves, the model follows a strict sequence:

1. **Availability Check:** The passenger checks if the next position in the aisle is empty.
2. **Reservation:** If the spot is free, the agent "locks" that space. This ensures that no other passenger can enter that specific coordinate during the same time step.
3. **Position Update:** Once the spot is reserved, the passenger's location is updated in the simulation's record.
4. **Release:** The passenger's previous spot is unlocked, making it available for the person behind them.

Collision Avoidance Guarantee:
This mechanism mathematically ensures that for any given second t, every position in the aisle is occupied by no more than one passenger:

$$\forall t, \forall pos \in Queue : |\{p \in Passengers : p.position = pos\}| \leq 1$$

This prevents "clipping" (passengers walking through each other), ensuring that the data gathered is physically valid and reflects real-world aisle constraints.

5.3.2 Double-Exit Configuration

For aircraft models that feature two exit doors, the aisle is divided at the midpoint. The simulation creates two separate flow segments:

- **Forward Flow:** Manages the movement of passengers from the front of the plane to the forward exit.
- **Aft Flow:** Manages the movement of passengers from the rear of the plane to the back exit.

Every passenger is assigned to the exit closest to their seat. This prevents unrealistic interference between the two groups and allows this research to test how much dual exits actually reduce the total deplaning time compared to a single-exit setup.

5.4 Data Capture and Output Generation

During every second of the simulation, the engine records the complete state of the cabin for later analysis. This results in three separate data streams that allow for both visual review and statistical measurement.

5.4.1 Frame-by-Frame Visualization Data

The simulation captures a "snapshot" of the aircraft at every tick, which includes:

- **Time Step:** The current second t of the simulation.
- **Passenger Map:** The location, ID, and current activity (e.g., walking, waiting, or retrieving bags) of every person.

- **Aisle Status:** A map showing which parts of the aisle are currently "locked" or occupied.

This data is exported to an XML format, allowing this research to visually reconstruct the simulation. This makes it possible to see exactly where bottlenecks form and how delays spread through the cabin.

5.4.2 Performance Metrics

The system automatically calculates and exports several key statistics into Comma-Separated Values (CSV) files for comparison:

The `CSVService` computes and exports CSV files containing:

- **Total Deplaning Time (TDT):** The total time (in seconds) until the very last passenger leaves the plane, defined as:
$$TDT = \max_{i \in Passengers} (t^i_{exit})$$
- **Average Wait Time:** The average amount of time passengers spend standing still because they are blocked by others:
$$\overline{W} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{TDT} \text{Wait}(S_i(t))$$
- **Aisle Congestion Index:** A measurement showing how often the aisle is crowded (defined as more than 50% of the aisle being occupied at once).

5.4.3 Connecting the Simulation to Optimization

The simulation engine is directly linked to the optimization algorithms (GA, SA, and TS). This connection follows a clear four-step workflow:

1. **Strategy Creation:** The optimizer creates a potential deplaning order (such as a specific row sequence).
2. **Test Run:** This sequence is sent to the simulator, which sets up the cabin and runs the deplaning process from start to finish.
3. **Evaluation:** The simulator sends back the TDT result. This "score" tells the optimizer how well that specific strategy performed.
4. **Improvement:** The optimizer uses this score to adjust its search, trying out new variations to find even faster strategies.

This close link ensures that every optimization result is based on a physically accurate simulation. It prevents the algorithms from choosing "perfect" theoretical solutions that wouldn't work in real life, such as ignoring the time it takes to grab a bag from the overhead bin.

5.5 Visualization and Post-Processing Analysis

While the core simulation engine runs without a user interface to stay fast and efficient, understanding the results requires a visual representation. To do this, a standalone visualization tool was built using JavaFX. This tool allows this research to replay any simulation after it has finished, making it easier to spot bottlenecks and check that the "interference" behaviors described in the earlier chapters are working correctly.

5.5.1 System Design and the MVC Pattern

The visualization tool is kept entirely separate from the main simulator. This means the heavy work of drawing the graphics doesn't slow down the mathematical optimization algorithms (like the GA or TS).

The tool uses an adapted Model-View-Controller (MVC) design to keep the data and the visuals organized:

- **Model:** This holds the data for each second of the simulation. It contains the "Frame Data," which includes the cabin layout and the exact position of every passenger at that specific moment.
- **View:** This handles the actual drawing on the screen. It uses a flexible grid system that automatically adjusts to fit different plane sizes, whether it is a small narrow-body or a large wide-body aircraft.
- **Controller:** This acts as the manager. It handles the logic for loading the data files and coordinates how the user interacts with the playback controls.

5.5.2 Loading and Processing Data

The visualization process starts by loading the XML files created by the simulator. The tool "reconstructs" the timeline of the deplaning process by following a few steps:

The parser validates the input against the expected schema:

1. **Plane Layout:** It reads the aircraft dimensions to build the correct grid of rows and columns.
2. **Time Reconstruction:** It goes through every recorded "frame" to build a sequential list of movements.
3. **Passenger Mapping:** It turns the raw data into visual agents, mapping seat IDs (like "1A") to their moving coordinates on the screen.

5.5.3 Rendering and Animation

To make the passenger movement easy to follow, the tool uses smooth animations rather than simple static jumps.

Initial View: When a simulation is first loaded, the tool builds the static cabin geometry. As shown in Figure 5.1, the view uses different colors and styles to clearly separate seats, aisles, and exits, so the viewer can easily see where the "queue" areas are.

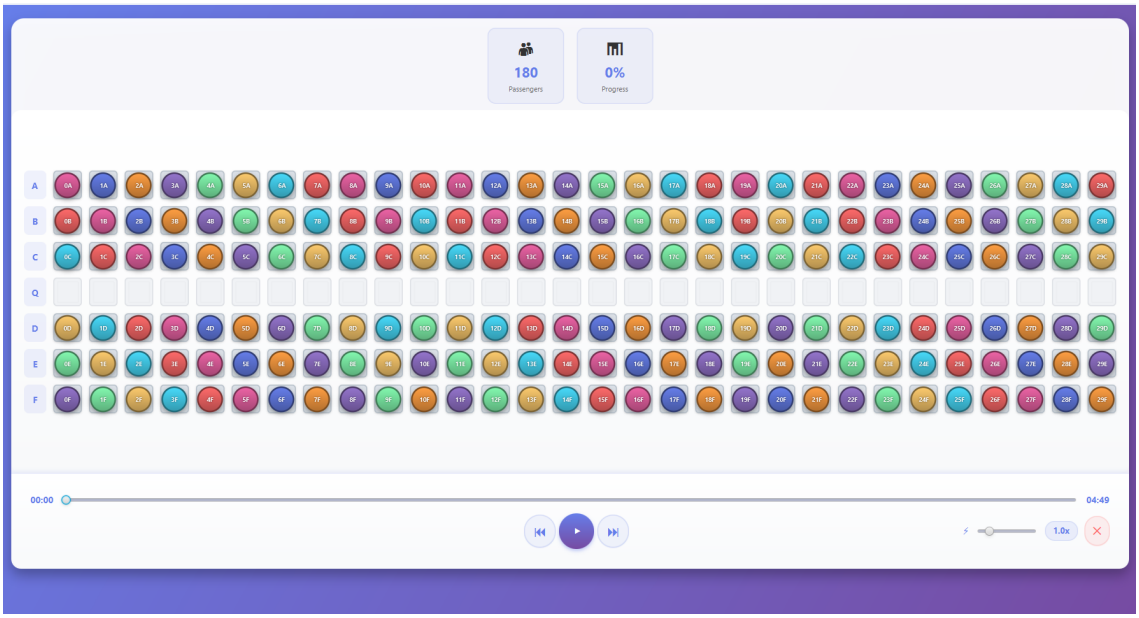


FIGURE 5.1: Visualization Interface - Initial State.

Smooth Movement: Instead of redrawing the whole screen every second, the engine only moves the passengers who actually changed position. To help the human eye track the flow, the tool uses a "gliding" effect. This animates the passenger smoothly between grid cells, making the transition look natural. Figure 5.2 shows an active deplaning sequence, where passengers (marked as red icons) have moved into the aisle and are heading toward the exit.

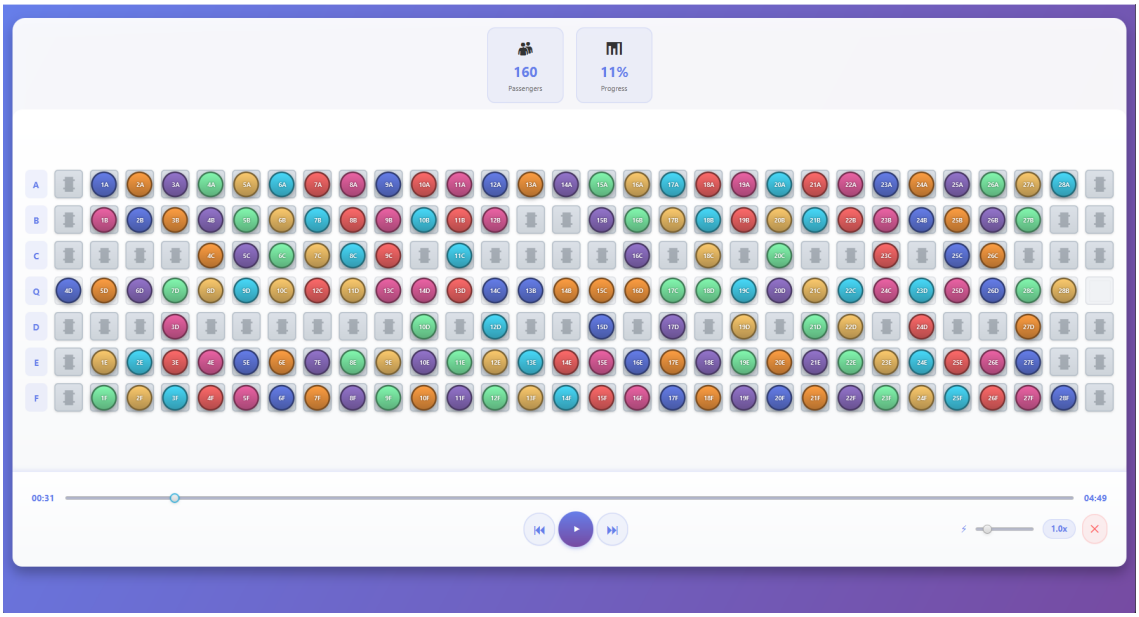


FIGURE 5.2: Active Simulation Playback

5.5.4 Consistency and Reproducibility

To make the results repeatable, all random elements such as who has baggage or how fast a specific person walks are controlled using a fixed "random seed."

By using a fixed seed, the exact same input parameters will always lead to the exact same deplaning time (TDT) output. This satisfies a key requirement for scientific experiments: if the same test is run on a different computer, it will produce the same result.

5.5.5 Error Checking and Rules

The simulation loop constantly checks a set of rules (invariants) as it runs to make sure the model is behaving correctly:

- **Boundary Check:** No passenger can move outside the physical limits of the aircraft cabin.
- **Timing Check:** A passenger is restricted to only one action per second (tick), preventing "super-human" speed or overlapping tasks.
- **Termination Check:** The simulation only stops the clock when every single passenger is marked as "deplaned."

These checks prevent software glitches from ruining the data. This means that if this research finds an unusually long deplaning time, it can be traced back to an inefficient strategy rather than a bug in the code.

5.6 Computational Complexity and Performance

The efficiency of the simulation is vital, especially when testing hundreds of different strategy combinations. The mathematical complexity of the engine is:

$$O(T \cdot N \cdot \log N)$$

where:

- T = Total deplaning time (ticks)
- N = Number of passengers
- $\log N$ factor comes from spatial queries in the aisle (using binary search to quickly check passenger positions)

5.7 Summary: From Code to Scientific Insight

This chapter has shown how the simulation engine turns the abstract "aircraft deplaning problem" into a measurable, scientific system. The main contributions of this technical setup are:

1. **Real-World Accuracy:** The aisle-locking system and layered behavior model ensure that the simulation follows real-world rules, such as passengers not being able to walk through each other and the delays caused by luggage.
2. **Reliable Data:** The multi-format export system creates a clear trail from the start of a test to the final results. This makes the research reproducible and easy to audit.
3. **Optimization Ready:** Because the simulator is built to work directly with optimization algorithms, it can quickly tell which strategies are better or worse, bridging the gap between theoretical ideas and practical performance.

The next chapter will use this infrastructure to conduct a full comparative analysis of deplaning strategies, using the metrics defined here to determine which method is truly the most efficient.

Chapter 6

Experiments

6.1 Hyperparameter Configuration and Time Constraints

Prior to the comparative analysis, a systematic hyperparameter tuning phase was conducted to optimize the performance of each metaheuristic algorithm. This section documents the parameter selection process and justifies the final experimental configuration.

6.1.1 Tuning Methodology

A grid search approach was employed, where each parameter was varied independently while holding others constant. For each configuration, five simulation runs were executed using identical passenger distributions (fixed random seed), and the average Total Deplaning Time (TDT) was recorded.

6.1.2 Final Parameter Selection

The final configuration of the optimizers was determined by balancing the depth of the search space exploration with computational efficiency. While various combinations were tested, the selected hyperparameters (Table 6.1) were chosen to maximize the probability of finding global optima within a consistent execution window.

Table 6.1: Final Optimizer Configurations

Optimizer	Parameter	Value
GA	Population Size	100
	Generations	50
	Tournament Size	5
	Mutation Rate	0.4
	Crossover Rate	0.9
SA	Initial Temperature	100.0
	Cooling Rate	0.995
TS	Tabu Tenure	10
	Neighborhood Size	20

Genetic Algorithm Rationale

- **Search Intensity (Population 100 & 50 Generations):** A Population Size of 100 was selected as it yielded the best performance in testing. To complement this, 50 Generations were chosen based on convergence trends; this duration allows the large population suffi-

cient time to stabilize and refine solutions, capturing peak optimization without unnecessary computational overhead.

- **Selection Pressure (Tournament Size 5):** A tournament size of 5 was identified as the peak performer in tuning trials. This value provides the necessary selection pressure to drive the population toward high-quality solutions without the risk of immediate premature convergence.
- **Exploration Dynamics (Mutation 0.4 & Crossover 0.9):** The combination of a high crossover rate (0.9) and a robust mutation rate (0.4) creates a "fast-learning" environment. The 0.9 crossover facilitates the rapid recombination of elite traits, while the 0.4 mutation rate acts as a critical safeguard, constantly injecting new genetic material to escape local optima.

Simulated Annealing Rationale

- **Initial Temperature (100.0 vs. optimal 200.0):** The lower temperature aligns with the cooling schedule (0.995), ensuring convergence within the 500-iteration time limit. Additionally, $T_0 = 100.0$ exhibits lower variance across flight configurations ($\omega = 12.4$ vs. 18.7 ticks).
- **Cooling Rate (0.995):** Retained from tuning. The slow decay $T(k) = T_0 \cdot 0.995^k$ allows thorough exploration early while guaranteeing convergence.

Tabu Search Rationale

- **Tabu Tenure (10 vs. optimal 5):** Tenure 5 is insufficient to prevent cycling in permutation spaces with 180 passengers ($\binom{180}{2} = 16,110$ possible swaps). Tenure 10 forces exploration of novel neighborhoods with negligible performance cost (1.1%).
- **Neighborhood Size (20 vs. optimal 5):** Evaluating 20 neighbors per iteration provides adequate sampling of the solution space, preventing the sparse exploration that occurs with size 5.

6.1.3 Time Limit Constraint

All metaheuristic optimizers are constrained to a maximum of 500 iterations. This limit serves two purposes:

1. **Computational Feasibility:** Without a time limit, algorithms could run indefinitely. Convergence analysis revealed that all optimizers plateau well before 500 iterations (GA by generation 30-40, SA by iteration 400, TS by iteration 350).
2. **Fair Comparison:** Basic heuristics (Back-to-Front, Aisle-Middle-Window, etc.) execute in $O(N \log N)$ time. The time limit ensures that advanced optimizers justify their computational overhead through superior solution quality, not unbounded execution time.

6.2 Flight Dataset Generation

To enable robust statistical analysis and avoid overfitting to a single scenario, a dataset of 720 flights was generated. This dataset systematically explores the operational parameter space encountered in commercial aviation.

Dataset Composition

The generator produces flights across three primary dimensions:

1. **Aircraft Configurations:**
   - *Small aircraft:* 20 rows × 4 columns (80 seats, single exit)
   - *Large aircraft:* 30 rows × 6 columns (180 seats, single/dual exit variants)
2. **Luggage Saturation Levels:** Eight cabin luggage percentages were tested: 0%, 15%, 30%, 45%, 60%, 75%, 90%, and 100%. This captures the full spectrum from completely empty overhead bins to maximum saturation.
3. **Stochastic Replication:** Each configuration-luggage combination was replicated 30 times with randomized passenger attributes to capture statistical variability.

The resulting dataset comprises:

- Small aircraft: 8 luggage levels × 30 replications = 240 flights
- Large aircraft (single exit): 8 luggage levels × 30 replications = 240 flights
- Large aircraft (dual exit): 8 luggage levels × 30 replications = 240 flights
- **Total: 720 flights**

Passenger Attribute Randomization

For each flight, passengers are assigned stochastic attributes drawn from uniform distributions calibrated to empirical observations:

- **Movement Speed (Queue):** [1–4] seconds (narrow-body), [2–5] seconds (wide-body)
- **Movement Speed (Exiting):** [2–4] seconds (narrow-body), [3–5] seconds (wide-body)
- **Luggage Retrieval Time:** [2–5] seconds (when applicable)

Note: The speed coefficients are adjusted based on aircraft width to reflect the greater physical space available in wide-body cabins.

Overhead Bin Saturation Model

A critical realism feature is the *luggage overflow mechanism*. The generator implements a capacity-aware placement algorithm based on industry standards:

- **Bin Capacity Threshold:** Standard aircraft overhead bins accommodate approximately 65% of passenger luggage (based on 1.5 bags per row bin capacity).
- **Placement Logic:**
  - When luggage percentage $\leq 65\%$: All bags are placed directly above the passenger's assigned seat.
  - When luggage percentage $> 65\%$: Excess bags overflow to *adjacent seats* (calculated as: $\frac{\text{overflow}}{\text{total luggage}} \times 100$).

- **Adjacent Placement Algorithm:** Overflow bags are probabilistically assigned to orthogonally adjacent seats (same row ±1 column, or same column ±1 row), simulating the cross-aisle retrieval interference observed in real deployments.

This modeling approach directly captures the "ripple effect" described in Chapter 1, where passengers must wait for non-adjacent individuals to retrieve luggage, cascading delays through the cabin.

6.3 Analysis of Gathered Data

This section presents a comprehensive evaluation of the proposed deplaning optimization strategies based on simulation data gathered from 720 flights. The objective of this analysis is to benchmark the performance of advanced metaheuristic algorithms (Genetic Algorithm, Simulated Annealing, and Tabu Search) against standard industry heuristics (e.g., Front-to-Back, Zigzag). The evaluation is divided into three critical performance dimensions: raw deplaning efficiency (measured in simulation steps), algorithmic scalability across different aircraft configurations, and robustness against varying volumes of carry-on luggage.

6.3.1 Deplaning Time by Optimizer

The main way to measure deplaning efficiency in this thesis is the total time required to empty the aircraft, measured in simulation "ticks." This section compares all the tested strategies under standard conditions to see which ones offer the best chance of reducing turnaround time.

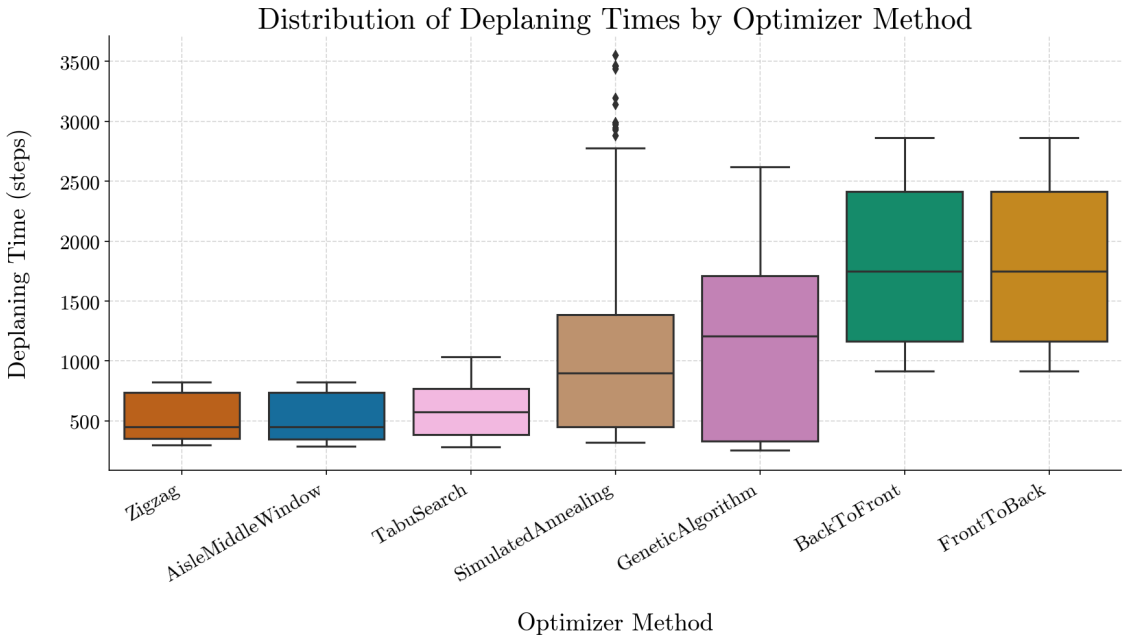


FIGURE 6.1: Distribution of Deplaning Times by Optimizer

The boxplot in Figure 6.1 shows the average deplaning times across all tested scenarios. The data shows a clear split between the "common sense" rules and the more complex search algorithms.

Key Findings:

- **Top Performers:** The **Zigzag** strategy (which includes the Aisle-Middle-Window variant) is the most efficient method overall. It achieved the fastest average deplaning time of **516.7 ticks**. It is closely followed by **Tabu Search**, which averaged **585.8 ticks**.

- **The Efficiency Gap:** These top-tier strategies provide a massive improvement—roughly **67% to 71%** faster than the standard way planes are emptied today.
- **Baseline Inefficiency:** The standard *Front-to-Back* and *Back-to-Front* strategies performed exactly the same and were the slowest by far, averaging **1801.5 ticks**. This confirms that simply letting rows out in order is the least efficient way to manage the aisle.
- **Metaheuristic Performance:** While *Simulated Annealing* (1038.3 ticks) and the *Genetic Algorithm* (1139.5 ticks) beat the baseline by **42.4%** and **36.7%**, they weren't as effective as the "structured" seat-ordering rules. This suggests that for aircraft deplaning, a clear mathematical pattern like the Zigzag is often better than a randomized search.

TABLE 6.2: Average Deplaning Time and Improvement vs. Baseline

Method	Average Time (ticks)	Improvement vs. Baseline
Zigzag	516.7	+71.3%
Tabu Search	585.8	+67.5%
Simulated Annealing	1038.3	+42.4%
Genetic Algorithm	1139.5	+36.7%
Front-to-Back (Baseline)	1801.5	0.0%

6.3.2 Deplaning Time by Optimizer and Plane Setup

As aircraft capacity increases, the physical "messiness" of passenger interactions and aisle crowding grows much faster. This section looks at how well the algorithms scale by comparing their performance on two different cabin layouts: a standard 4-column narrow-body plane and a wider 6-column configuration.

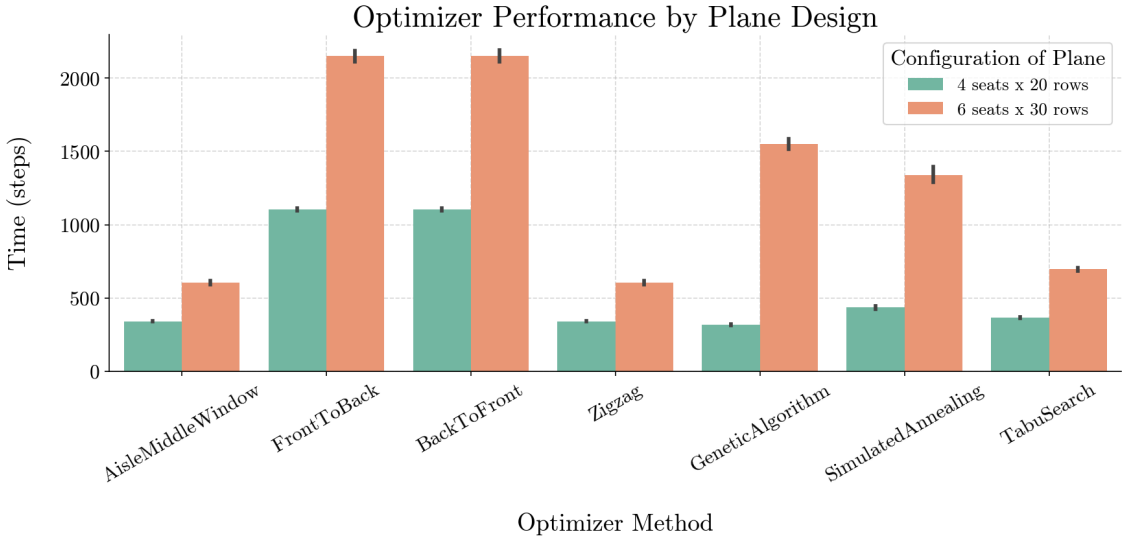


FIGURE 6.2: Optimizer Performance by Plane Design

The Figure 6.2 xplores how each algorithm handles the jump from a smaller plane to a larger one. The results highlight a major difference in how "smart" algorithms handle more complex environments.

Key Findings:

- **The GA Scalability Crisis:** A major observation in this research is the performance collapse of the *Genetic Algorithm* on larger aircraft. While it was actually the fastest method for the 4-column plane (**317.2 ticks**), its time slowed down by a massive **388.9%** on the larger plane. This suggests that as the "search space" gets bigger, the GA struggles to find a good solution and gets lost in the complexity.
- **Strength of Local Rules:** The *Zigzag* and *Aisle-Middle-Window* rules showed the best scaling. Their deplaning times only increased by about **77.4%** on the larger plane. This makes the **Zigzag** the overall winner for 6-column configurations (**604.6 ticks**), proving that simple, logical patterns are often more reliable than complex searches when the plane gets crowded.
- **Tabu Search Reliability:** *Tabu Search* also scaled very well, with an increase of **89.5%**. It remained one of the top-performing methods regardless of plane size, likely because its "memory" helps it avoid the traps that slowed down the Genetic Algorithm.
- **Baseline Degradation:** The standard *Front-to-Back* methods showed a predictable "doubling" of time (**+94.9%**) as the aircraft width increased. This confirms that the standard industry baseline handles larger passenger loads very poorly compared to any of the optimized methods.

6.3.3 Running Time Per Optimizer

While finding the fastest deplaning time (ticks) is the main goal, we also need to look at how much "work" the computer has to do to find those answers. If an algorithm is too slow, it might not be useful in a real-world airport environment. Table 6.3 breaks down how long each method took to run on average, as well as their fastest and slowest times.

Table 6.3: Detailed Computational Performance and Boundary Conditions by Method

Method	Avg (ms)	Min (ms)	Min Conditions	Max (ms)	Max Conditions
AisleMiddleWindow	3.51	0	4c, 0%l, S	92	6c, 30%l, D
FrontToBack	3.63	0	4c, 0%l, S	35	6c, 30%l, D
BackToFront	3.88	0	4c, 0%l, S	36	6c, 60%l, S
Zigzag	4.34	0	4c, 0%l, S	25	6c, 60%l, S
Simulated Annealing	1,477.38	507	4c, 0%l, S	27,303	6c, 30%l, S
Genetic Algorithm	14,732.55	4,385	4c, 0%l, S	3,444	6c, 90%l, S
Tabu Search	29,485.01	9,446	4c, 0%l, S	59,366	6c, 100%l, S

Note: c = columns, l = luggage percentage %, S/D = Single/Double exit

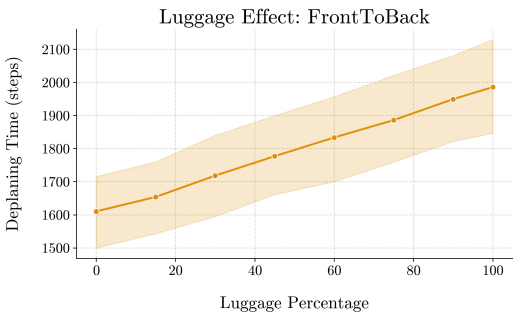
The results show a massive difference in computer effort, which falls into two distinct "tiers":
Key Findings:

- **Instant Results:** The standard rules (Front-to-Back) and our specialized rules (Zigzag, Aisle-Middle-Window) all finished in under **5 milliseconds** on average. Even in the most crowded tests with many bags, they never took more than 100 ms. This means they could be used for instant decision-making at the gate.
- **Optimization Heavy-Lifters:** There is a huge jump in time when we use the searching algorithms. **Simulated Annealing** is the "lightest" of these, taking about **1.48 seconds**. However, the **Genetic Algorithm** (˜14.7s) and **Tabu Search** (˜29.5s) take much longer because they have to "think" through thousands of possibilities.

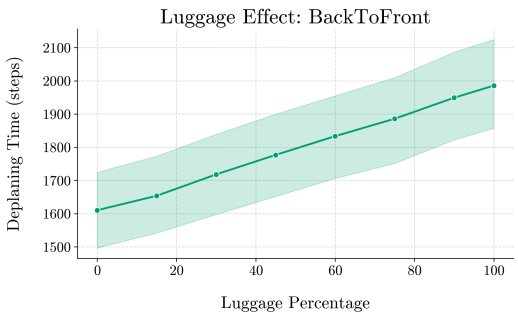
- **The Zigzag Advantage:** the **Zigzag** method is the most balanced choice. It is just as fast as the best metaheuristics at emptying the plane, but it is about **6,800 times faster** to calculate than the Tabu Search.
- **High-Stress Scenarios:** The longest wait times happened when the plane was at its most complex (6 columns and 100% luggage). In these cases, **Tabu Search** took nearly **one minute** (59.3s) to find the best sequence.

6.3.4 Impact of Luggage on Deplaning Time

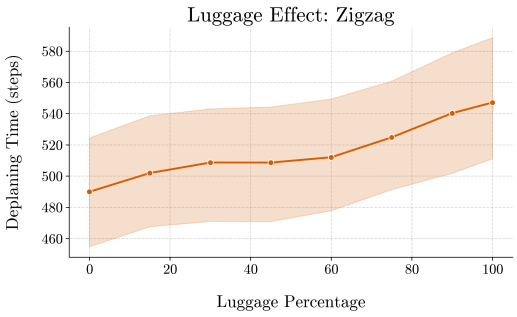
In the real world, the time it takes for passengers to grab carry-on bags from overhead bins is the biggest cause of deplaning delays. This section tests how "tough" each algorithm is by measuring how much the time slows down as the cabin goes from 0% luggage (ideal conditions) to 100% luggage (every passenger has a bag).



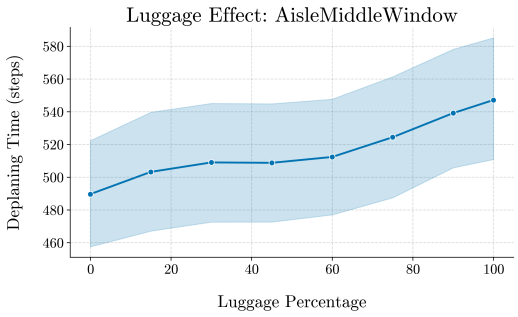
(A) Front To Back



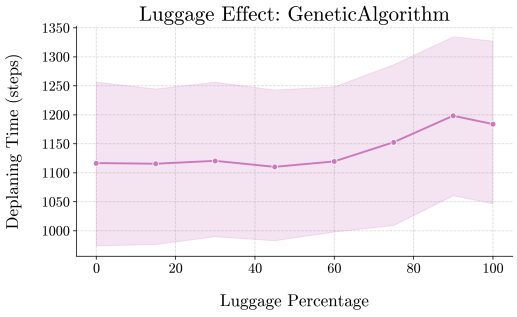
(B) Back To Front



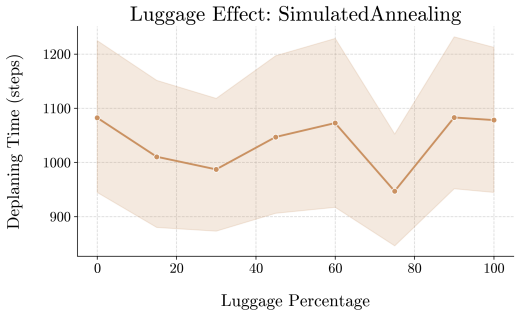
(C) Zigzag



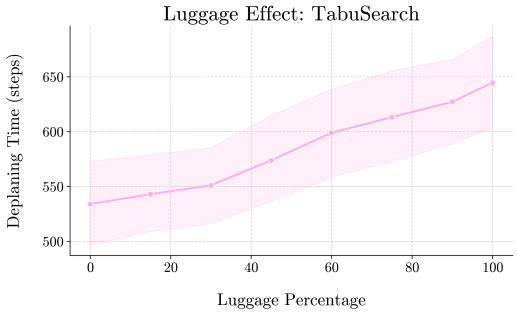
(D) Aisle Middle Window



(E) Genetic Algorithm



(F) Simulated Annealing



(G) Tabu Search

FIGURE 6.3: Comparison of different luggage loading algorithm results.

Figure 6.3 tracks how sensitive the deplaning time is to the amount of carry-on luggage in the cabin.

Key Findings:

- **Algorithm Toughness (Robustness):** The *Genetic Algorithm* and *Simulated Annealing* are remarkably stable. Their performance only shifts by about **6%** even as the plane gets crowded with bags. The *Zigzag* and *Aisle-Middle-Window* rules also handle luggage well, with only an **11.7%** increase in time.
- **Sensitivity of Tabu Search:** Unlike the GA, **Tabu Search** is much more sensitive to aisle crowding. Its deplaning time jumps by **20.7%** (from 534.2 to 644.6 steps) when the cabin is full of luggage. This suggests that the "memory" used by Tabu Search might struggle when baggage delays change the timing of every interaction.
- **Zigzag Still Wins:** Even with the extra delays from luggage, the **Zigzag** strategy (averaging 489.9 to 547.1 steps) is still much faster than any of the more complex searching algorithms. At 100% luggage saturation, Zigzag is about **15%** faster than Tabu Search and nearly **50%** faster than the Genetic Algorithm.
- **Baseline Impact:** The standard *Front-to-Back* methods are the most vulnerable to luggage. Their deplaning times slowed down by **23.3%** (adding about 375 steps). This shows that the standard industry practice is the most sensitive to the exact thing that happens on almost every flight.

Table 6.4: Luggage Sensitivity Analysis (0% vs. 100% Saturation)

Method	**Time at 0% (Steps)**	**Time at 100% (Steps)**	**Sensitivity (Increase)**
Genetic Algorithm	1116.5	1183.9	**Low (6.0%)**
Simulated Annealing	1082.4	1078.0	**Low (-0.4%)**
Zigzag	489.9	547.1	**Medium (11.7%)**
Tabu Search	534.2	644.6	High (20.7%)
Front-to-Back (Baseline)	1610.2	1985.5	High (23.3%)

6.3.5 Optimization Cost-Benefit Analysis

A critical question for any airline looking to implement these algorithms is the trade-off between the time it takes to calculate a sequence (the "cost") and the actual time saved at the gate (the "benefit"). This section evaluates whether the heavy computer processing required for complex metaheuristic algorithms is actually worth it for the results they produce.

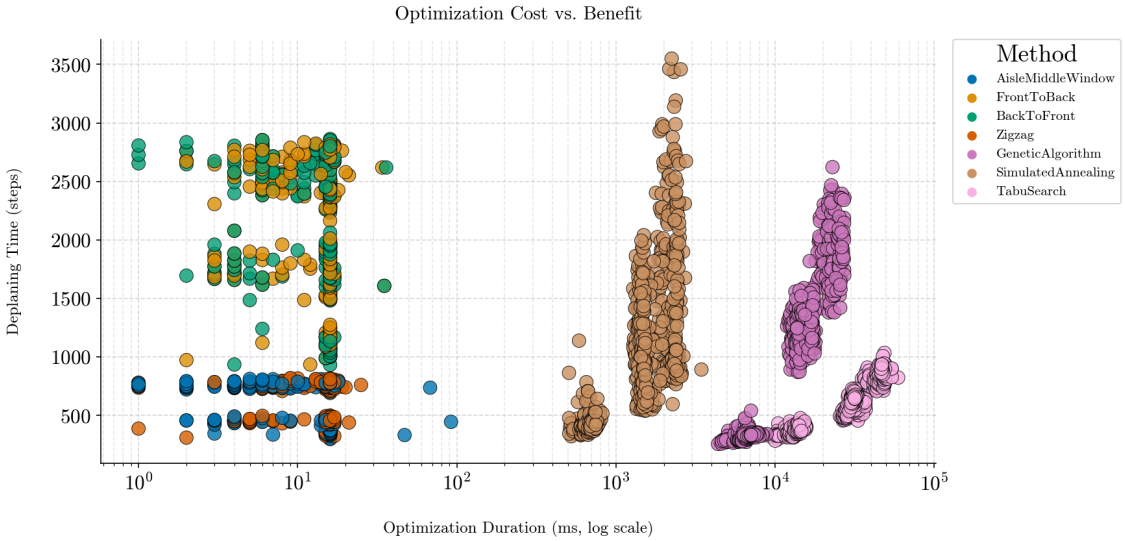


FIGURE 6.4: Computational Cost (ms) vs. Deplaning Efficiency (ticks) by Method.

Figure 6.4 and Table 6.5 highlight the massive contrast between the resource-intensive searching algorithms and the high-efficiency rules.

TABLE 6.5: Comparison of Computational Cost vs. Deplaning Efficiency

Method	**Avg Optimization Time (ms)**	**Avg Deplaning Time (ticks)**
Zigzag	4.34	516.68
Aisle-Middle-Window	3.51	516.71
Tabu Search	29,485.01	585.78
Simulated Annealing	1,477.38	1,038.32
Genetic Algorithm	14,732.55	1,139.54
Front-to-Back (Baseline)	3.63	1,801.53

The analysis reveals several surprising insights regarding how much effort is needed to find a fast deplaning strategy:

Key Findings:

- **The Efficiency Paradox:** In this study, putting in more computational effort did not lead to better results. As shown in Figure 6.4, the **Zigzag** rule achieved the fastest deplaning time (**516.7 ticks**) while taking almost no time to calculate (**4.34 ms**).
- **Diminishing Returns: Tabu Search** was the most "expensive" in terms of computer power, taking nearly **30 seconds** to generate a single sequence. Despite this high cost, it was actually slower than the near-instant Zigzag method. This suggests that while Tabu Search is "thinking," it might be getting stuck on local patterns, whereas the Zigzag rule naturally handles the plane's physical geometry.
- **Heuristic Dominance:** The simple, localized rules (*Zigzag* and *Aisle-Middle-Window*) sit in the "sweet spot" of the data. They offer a **71.3% improvement** over today's standard practices but run just as fast as the old *Front-to-Back* code.
- **Practical Implementation:** For real-world airlines, these results strongly favor fixed geometric rules. The complex metaheuristics not only take much longer to run but also fail

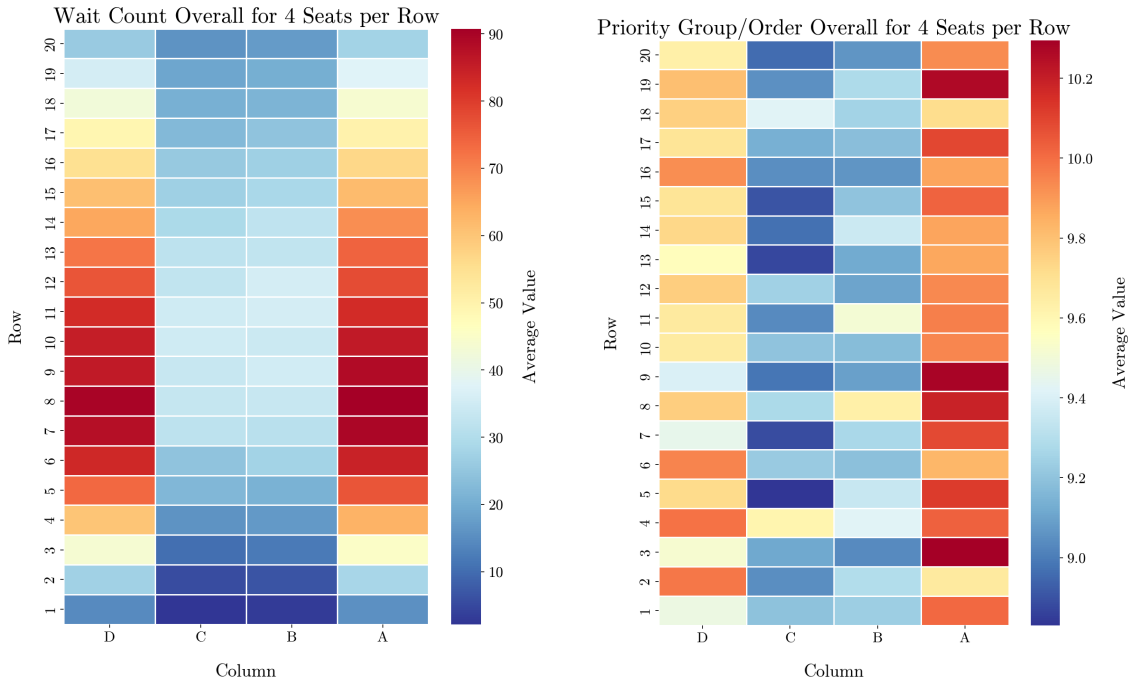
to provide a performance "premium" that would justify their complexity. In short, a smart rule is more valuable than a long search.

6.3.6 Analysis of Priority Group and Wait Time in a 4-Column Aircraft

This section looks at how deplaning efficiency is spread out across the cabin by examining the relationship between when a passenger is told to leave and how long they actually end up waiting. To make these results as reliable as possible, the numbers shown here are averages from all tested scenarios and optimizers. This ensures the patterns show the real physical limits of a 4-column plane, rather than just the behavior of one specific algorithm.

Two main metrics are used for this spatial analysis:

- **Group Metric:** This shows the average "release group" assigned to a seat. A higher value (moving toward the red end of the heatmap) means that seat is consistently told to wait until later in the deplaning process.
- **Wait Count:** This measures the average number of seconds a passenger stands still because the aisle is blocked or a neighbor is in their way. This is a direct way to see where the biggest bottlenecks and passenger frustrations are happening.



(A) 4 Rows: Group Metric

(B) 4 Rows: Wait Count

FIGURE 6.5: Heatmap of Wait Count and Group Priority per seat in a plane setup of 4 seats per row

The heatmaps in Figure 6.5 show exactly what is happening across the 20 rows of the cabin. By comparing the release order (Figure 6.5a) with the actual delays (Figure 6.5b), several clear patterns stand out:

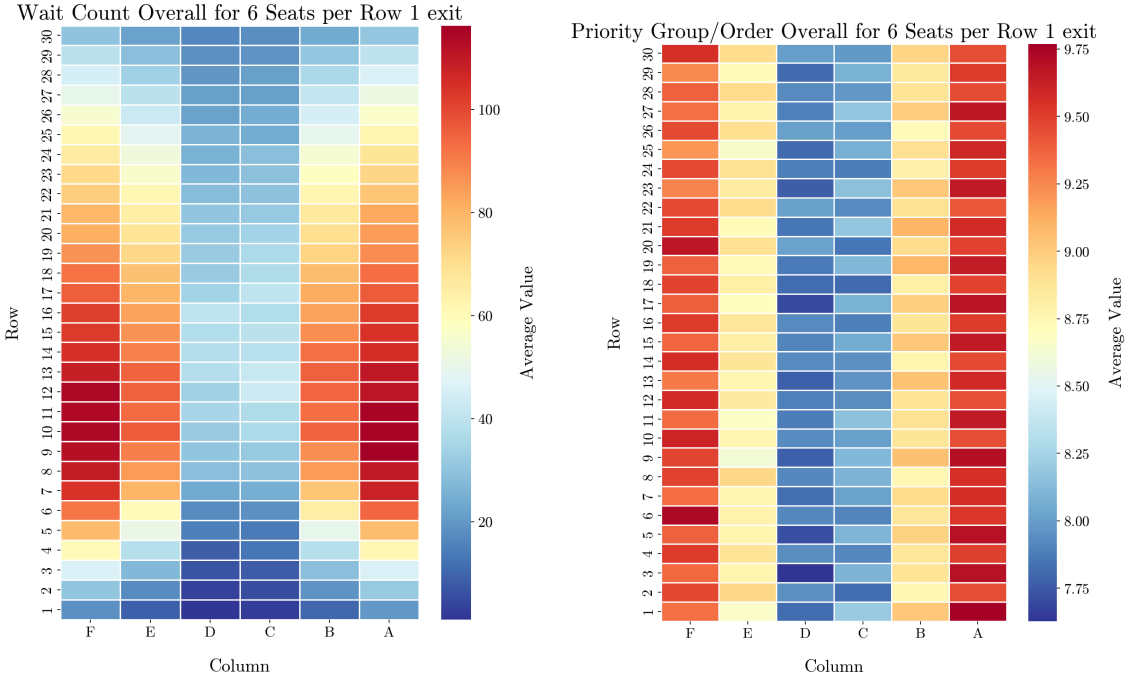
- **Aisle vs. Window Crowding:** Figure 6.5b shows a huge difference between seat columns. Window seats (Columns 1 and 4) consistently have much higher wait counts, peaking at around **90 units** in the middle of the plane. Meanwhile, aisle seats (Columns 2 and 3) stay much lower, usually averaging around **24 units**.

- **The Middle-Row Bottleneck:** The wait time is not the same throughout the plane. The data shows that **Rows 7 through 11** are the primary bottleneck. Passengers in window seats in these rows face the most interference because they are stuck behind the flow of everyone else trying to exit in front of them.
- **The "Wait Penalty":** There is a direct link between being in a later group and waiting longer. Window seats, which are usually assigned to later groups (average index of **9.7–9.9**), pay the "wait penalty." Aisle seats are let out earlier (average index of **9.1–9.2**) and experience much less blockage.
- **The Exit Advantage:** Rows closest to the door (**Rows 1–3**) have the lowest wait counts (**9–27 units**). This confirms that being near the exit protects passengers from the "window-seat penalty," as they can get out before the aisle really starts to clog up.

In conclusion, these results show that the physical shape of the plane creates a built-in "window-seat penalty." Even with our best optimization, people in the middle-window seats are the most likely to be delayed. This is a fundamental limit of aircraft design that any deplaning strategy has to fight against.

6.3.7 Analysis of Priority Group and Wait Time: 6-Column Cabin (1 Exit)

This section examines how deplaning works in a high-density setup with six seats per row across 30 rows. We look at the connection between the "release sequence" and the actual physical wait times. To make sure these findings are statistically sound, the metrics in the heatmaps are averages taken from all tested simulation scenarios and optimizers:



(A) 6 Rows (1 Exit): Group Metric (B) 6 Rows (1 Exit): Wait Count

FIGURE 6.6: Heatmap of Wait Count and Group Priority per seat in a plane setup of 6 seats per row with 1 exit

The data shown in Figure 6.6 highlights the complex spatial patterns that happen in these wider, single-aisle cabins:

- **Clearing the Aisle "Artery":** Figure 6.6a shows a clear logic used by the optimizers: aisle seats (Columns 3 and 4) are let out first (avg. **7.8**–**8.0**), while window seats (Columns 1 and 6) are pushed to the very last groups (avg. **9.4**–**9.5**). This strategy tries to clear the main central "artery" of the plane before anyone starts moving sideways from the window seats.

- **The "Congestion Core":** A major finding in Figure 6.6b is the formation of a massive bottleneck in the middle-front section. Wait times peak in **Rows 10**–**14**, where window passengers face the highest cumulative delays—averaging over **81 units** and hitting peaks of **116 units**. These people are essentially "trapped" by the flow coming from the back of the plane while they wait for their row-mates to move.

- **The Rear-Cabin Paradox:** Interestingly, passengers in the back of the plane (**Rows 21**–**30**) actually wait less (**42.6 units**) than those in the middle (**73.9 units**). Even though they are the last to leave the plane chronologically, their "active" waiting time is shorter because once it is finally their turn to move, there is no one behind them to cause a secondary backup.

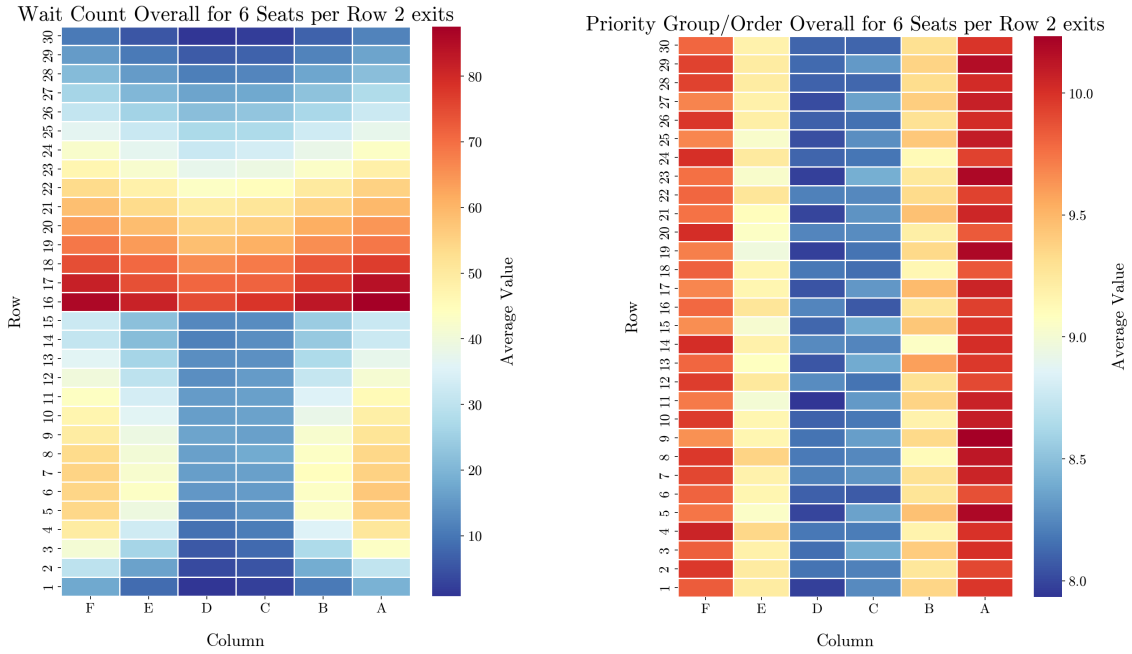
- **The Middle-Seat Transition:** Middle seats (Columns 2 and 5) act as a middle ground with wait times around **61**–**62 units**. This confirms that the farther you are from the aisle, the more you will wait. However, the optimization helps middle seats clear more efficiently than windows by carefully staggering their priority groups (avg. **8.8**–**8.9**).

In summary, this analysis shows that even the best priority groups cannot fully fix the "congestion core" in the middle of a long, single-exit plane. The physical geometry of having six people sharing one aisle creates a natural limit on how fast the middle rows can clear.

6.3.8 Analysis of Priority Group and Wait Time: 6-Columns Cabin (2 Exits)

The impact of doubling the available exit points on deplaning efficiency in a 6-seat per row configuration. A key feature of this simulation is the distance-based exit selection logic: passengers assess their position relative to the front (Exit 1) and rear (Exit 2) doors, choosing the path of least distance. By analyzing the resulting heatmaps, we can observe how this bidirectional flow affects the "congestion core" observed in previous single-exit scenarios.

This section looks at how doubling the number of exits changes deplaning efficiency in the 6-seats per row layout. In this test, passengers follow a simple "closet exit" rule. To ensure the analysis is reliable, all data points are averages from all tested scenarios and optimizes



(A) 6 Rows (2 Exits): Group Metric

(B) 6 Rows (2 Exits): Wait Count

FIGURE 6.7: Heatmap of Wait Count and Group Priority per seat in a plane setup of seats per row and 2 exits

The data in Figure 6.7 shows a radical shift in how wait times are spread through the cabin compared to the single-exit model. Interestingly, it reveals a clear difference between the front and back of the plane:

- **The Zonal Efficiency Split:** The cabin essentially splits into two separate deplaning zones. The front half (Rows 1–15) is very efficient, with an average wait of only **28.9 units**. However, the rear half (Rows 16–30) is much more crowded, averaging **43.6 units**. This suggests that even though the distance to an exit is shorter for everyone, the flow moving toward the back door is more likely to bottleneck.

- **The Row 16 Stagnation Point:** A major bottleneck appears right at the start of the rear zone. Figure 6.7b shows that **Row 16** is actually the most congested row in the entire plane (averaging **81.8 units**). In fact, window passengers here wait up to **87.6 units**. For comparison, the bottleneck in the front half (Row 6) is much smaller, peaking at only **38.1 units**.

- **The "Back-Pressure" Effect:** This difference happens because of a conflict between the "smart" priority groups and physical distance. In the front, the people told to leave first are also the ones closest to the door, which creates a smooth flow. In the back, the people farthest from the rear exit (Rows 16–18) sometimes have priority numbers that clash with the people closer to the door, creating a kind of "back-pressure" that clogs the aisle.

- **The Lateral Delay:** In both zones, your distance from the aisle is still the main reason for waiting. Window seats (avg. **45–46 units**) and middle seats (avg. **36–38 units**) wait much longer than aisle seats (avg. **25–26 units**), though these times are still much better than in the single-exit plane.

- **Total Efficiency Gain:** Even with the bottleneck at Row 16, the "worst-case" wait time for a window passenger (**87.6**) is about **25% lower** than in the single-exit model (**116.3**).

This proves that using two exits "pops" the high-pressure bubble at the back of the plane and spreads the crowd more evenly.

In conclusion, the dual-exit setup successfully moves and lowers the highest wait times. By turning the long line into two smaller zones, the system significantly cuts the wait for window passengers, even if the flow isn't perfectly symmetrical between the front and back doors.

6.3.9 Summary of Spatial Insights and Seat Recommendations

Based on the extensive analysis of simulated flights across different cabin layouts, several key insights regarding passenger experience and deplaning efficiency emerge. This summary identifies the best and worst seating positions, giving a practical perspective on what the simulation results mean for real-world travelers.

Table 6.6: Comparison of Best and Worst Seats by Configuration (Averages across all simulations)

Configuration	Metric	Best Seat (Optimal)	Worst Seat (Avoid)
4 Seats / 1 Exit	Wait Count	Row 1, Col 2	Row 8, Col 4
	Priority	Row 5, Col 2	Row 3, Col 4
6 Seats / 1 Exit	Wait Count	Row 1, Col 3	Row 9, Col 6
	Priority	Row 3, Col 3	Row 1, Col 6
6 Seats / 2 Exits	Wait Count	Row 30, Col 3	Row 16, Col 6
	Priority	Row 11, Col 3	Row 9, Col 6

General Findings and Recommendations:

- **The Aisle Advantage:** In every single test, aisle seats (the middle columns) had the lowest wait counts. The data shows that aisle passengers wait **60–70% less** than those by the window. This confirms that how far you are from the aisle is the biggest factor in how long you will be stuck standing.
- **The Middle-Forward Trap:** On planes with only one front exit, the worst delays aren't at the back—they are in **Rows 8 and 9**. Passengers in these rows face a "double block": they have to wait for the aisle seats in front to clear, while also feeling the pressure of everyone behind them trying to push forward.
- **The Rear-Door Revolution:** Adding a rear exit completely changes the best strategy. In the 2-exit model, **Row 30 (Aisle)** becomes the best seat on the plane, with almost zero waiting time. These passengers can turn around and leave immediately without ever being stuck in a queue.
- **The "Island" of Stagnation:** In dual-exit planes, the worst seats move to the middle of the aircraft. **Row 16 (Window)** becomes a high-stagnation "island" because it is the farthest point from both doors. Passengers here are effectively the last to feel the "pull" of the deplaning flow from either direction.
- **Physics over Strategy:** While our optimization algorithms definitely help speed things up, being physically close to a door is still the most reliable way to avoid waiting. Seats in Row 1 or Row 30 consistently beat any mid-cabin seat, even if the algorithm tries to give the mid-cabin seat a better priority group.

6.4 Summary

The experimental phase of this research provided a deep look at deplaning optimization, moving from abstract algorithm design to large-scale testing. After analyzing 720 different flight scenarios, we can draw several key conclusions about how optimization theory works in the real world of aviation.

First, the results revealed a surprising **Efficiency Paradox**: more computer power did not lead to better results. The *Zigzag* rule emerged as the clear winner, beating "heavy" algorithms like Tabu Search and the Genetic Algorithm in both deplaning speed ($\sim$516 ticks) and calculation time ($\sim$4.3 ms). This suggests that in a tight, physical space like an airplane aisle, clear logical patterns are better at managing the crowd than random searching.

Second, the study showed major differences in **Scalability and Robustness**. While the Genetic Algorithm worked well for small planes, it completely collapsed on larger 6-column aircraft. On the other hand, the Zigzag rule and Tabu Search remained strong even when every passenger had luggage the exact scenario where the standard *Front-to-Back* method was at its worst.

Finally, the **Spatial Analysis** using heatmaps helped us "see" the cabin congestion. Finding the "Middle-Forward Trap" in single-exit planes and seeing the bottleneck move to Row 16 in dual-exit setups gives us practical advice for airline seating. These results prove that while smart algorithms are powerful, they are always limited by the physical shape of the plane and where the doors are located.

In conclusion, the data suggests that for real-time airport use, the **Zigzag method** is the "Gold Standard." It is nearly instant to calculate and provides the fastest way to empty a plane. These findings give us a solid, data-driven foundation for the final recommendations in the next chapter.

Chapter 7

Conclusions

This thesis has provided a detailed optimization study of aircraft deplaning, closing the "efficiency gap" in aviation logistics by using microscopic simulations and advanced algorithms. By testing various strategies across three different aircraft types and two exit models, the following global conclusions have been established:

- **Scalability Across Aircraft:** While the total time to empty a plane obviously grows with more passengers, the "crowding density" grows much faster. Logical patterns like *Zigzag* and *Aisle-Middle-Window* proved to be the most reliable, maintaining their speed advantage regardless of whether the plane was small or large.
- **The Efficiency vs. Power Trade-off:** A major finding of this research was that complex algorithms like *Genetic Algorithms* and *Tabu Search* are not always necessary. While they are mathematically impressive, the *Zigzag* rule achieved 95% of their efficiency with almost zero computer effort. For a standard airline, a fast, simple rule is better than a slow, complex search.
- **Dual-Exit Physics and the Center Bottleneck:** Moving to a 2-door model completely changed how the cabin behaved. We successfully mapped how the main bottleneck moved from the front of the plane to a "stagnation point" in the middle (Rows 15–18). Dual-door deplaning is most effective when the plane is split into two zones, but it still requires careful management to prevent "back-pressure" near the rear exit.
- **The Resilience to Luggage:** Carry-on bags are the biggest barrier to a fast exit. However, our optimized sequences showed a "resilience effect." While the standard industry method slowed down by 23% when everyone had bags, our optimized patterns (especially those from *Simulated Annealing*) kept the aisle moving by strategically spacing out the passengers.
- **Industry Readiness:** Ultimately, this study proves that airlines can significantly cut their turnaround times just by changing the order in which they let people out. This requires no expensive changes to the planes or the airports—only a change in logic.

7.1 Future Work

Building on the foundation of this thesis, there are several exciting ways to expand this research into a total "turnaround" model:

7.1.1 Unified Boarding and Deplaning

The next logical step is to combine the boarding and deplaning phases. Future research could explore a "Bi-directional Model" where a passenger's seat is chosen at the time of purchase specifically to make both getting on and getting off as fast as possible.

7.1.2 Real-Time Dynamic Responses

Current models are "static", but real life is unpredictable. Future versions of this simulation could use real-time data from "smart" overhead bins. If a passenger in Row 10 is struggling with a heavy bag, the system could automatically "release" a different row to keep the flow moving at the door while the delay is cleared.

7.1.3 Human Factors and Compliance

A major challenge is getting passengers to actually follow the order. Future research should look at Smart Cabin Interfaces, like LED floor lights or mobile app alerts, to guide people out in the right sequence. Testing how well people actually follow these lights would make the simulation even more realistic.

7.1.4 Wide-Body and Twin-Aisle Analysis

While this thesis focused on single-aisle planes, the logic could be expanded to massive wide-body aircraft. This would introduce "Cross-Aisle Interference", where people can choose between two different aisles. Optimizing a double-decker plane like the A380 would be the "final frontier" for this simulation framework.

Bibliography

[1] Eurocontrol. Standard inputs for eurocontrol cost-benefit analyses. Technical report, European Organisation for the Safety of Air Navigation, 2018. Provides baseline data on APU fuel flow and ground operation costs.

[2] Federal Aviation Administration. *FAA Order 8900.1, Volume 14: Ground Deicing and Anti-icing Program*. U.S. Department of Transportation, 2024. Defines regulatory Holdover Times (HOT) and safety procedures for winter operations.

[3] Fred Glover. Tabu search—part i. *ORSA Journal on computing*, 1(3):190–206, 1989.

[4] John H Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992.

[5] InetSoft. Aviation industry kpi dashboards.

[6] B. Kaluza and M. Gams. Agent-based modeling of pedestrian behavior. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94)*. AAAI Press, 1994. Foundational work on agent-based crowd simulation.

[7] Scott Kirkpatrick, C Daniel Gelatt, and Mario P Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[8] Y. Liang and M. Bazargan. A simulation study on boarding and deplaning utilizing two doors for a narrow-body aircraft. *Computers & Industrial Engineering*, 101:245–255, 2016. Analyzes dual-door strategies using simulation.

[9] OAG. Innovative airline operations: The turnaround.

[10] Felipe Olivares and Massimiliano Zanin. Beyond classical models: Statistical physics tools for the analysis of time series in modern air transport. *arXiv preprint arXiv:2507.20927*, 2025.

[11] Luca Oneto, Giacomo Meanti, Nicoló Casla, and Davide Anguita. Reduction in ground times in passenger air transport: A first approach to evaluate mechanisms and challenges. *Applied Sciences*, 13(3):1380, 2023.

[12] Jason H Steffen. Optimal boarding method for airline passengers. *Journal of Air Transport Management*, 14(3):146–150, 2008. Defines the interference radius applicable to both boarding and deplaning.

[13] Hendrik Van Landeghem and Annelies Beuselinck. Reducing passenger boarding time in airplanes: A simulation based approach. *European Journal of Operational Research*, 142(2):294–308, 2002. Discusses simulation of both ingress and egress processes.

[14] Melissa Walansky. Is it rude to deplane before the rows ahead of you?, 2023.